

B.M.S. COLLEGE OF ENGINEERING BENGALURU
Autonomous Institute, Affiliated to VTU



Lab Record

Object-Oriented Modeling — 23CS5PCOOM

Submitted in partial fulfillment for the 5th Semester Laboratory

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by:

Darshan YG

1BM23CS087

Department of Computer Science and Engineering
B.M.S. College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019
August 2025-December 2025

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Object-Oriented Modeling(23CS5PCOOM)
laboratory has been carried out by Darshan YG(1BM23CS087) during the 5th
Semester August 2025-December 2025

NAME OF THE Your Batch Incharge and designation:

Sonika Sharma

Assistant Professor

Department of Computer Science and Engineering

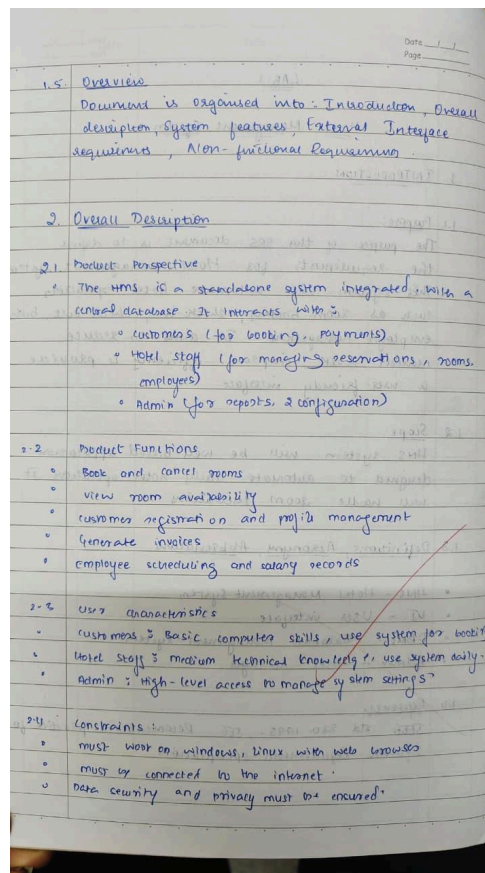
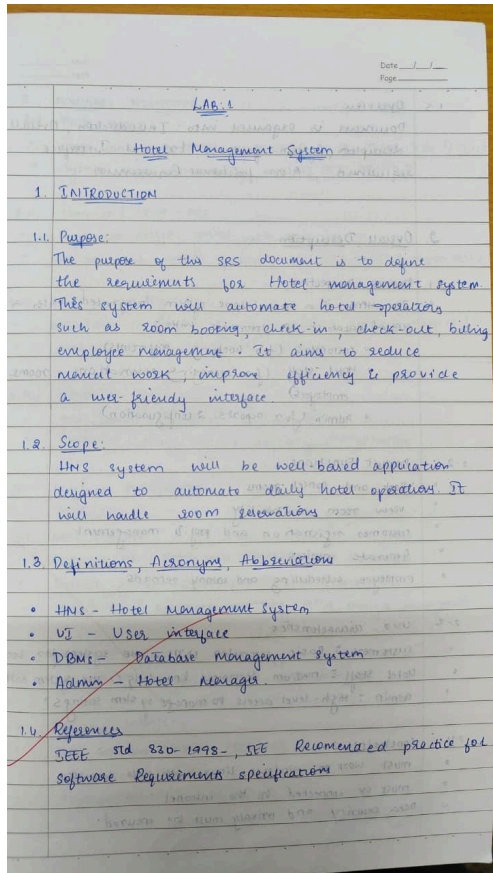
B.M.S. College of Engineering, Bangalore

Table of Contents

1. Hotel Management System	3-8
2. Credit Card Processing	9-16
3. Library Management System	17-25
4. StOck Maintenance System	27-34
5. Passport Automation System	35-42

1. Hotel Management System

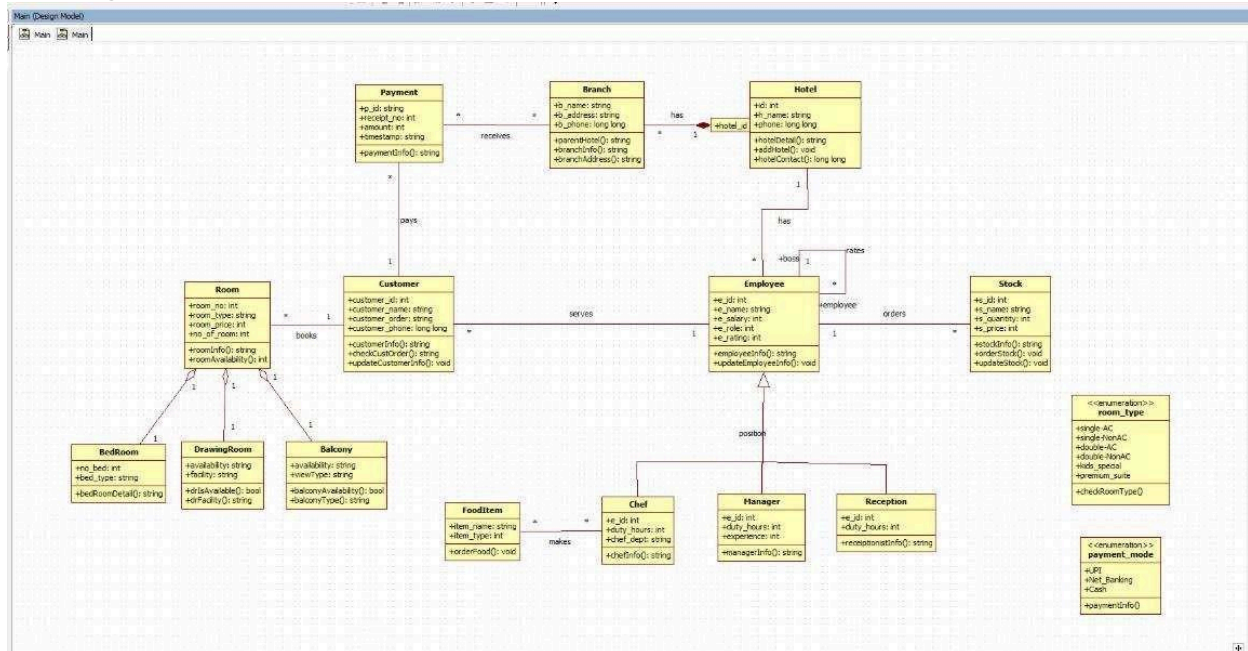
SRS Document



3.	FUNCTIONAL REQUIREMENTS
3.1	Room Booking
•	Users can book rooms by selecting type, date, no. of guests
•	System checks availability and confirms booking
3.2	Check-in / Check-out
•	Staff can assign rooms, update check-in / check-out
•	on front desk, system generates bill and updates room status
3.3	Billing and Payment
•	Automatic calculation of charges (room, services, taxes)
•	Integration with payment gateways
3.4	Employee management
•	Add, update, & remove employee records
•	Maintain salary details and duty assignments
4.	Interface Requirements
4.1	User Interface
•	Customer UI: Simple web interface for bookings
•	Staff UI: Dashboard for check-in / check-out
•	Admin UI: Advanced reporting and employee management
4.2	Hardware Interface
•	Runs on PC, laptop, or mobile with internet
4.3	Software Interface
•	Web browser (Chrome, Firefox)
•	Database (MySQL)
•	Payment Mtn (Paytm)

5.	Non-Functional Requirements
•	Performance: System should handle 1000 concurrent users
•	Security: Data encryption, secure login
•	Reliability: 99.9% uptime
•	Usability: Intuitive UI for customers & staff
•	Maintainability: Modular code, easy updates
•	Scalability: Support multiple hotels in future
6.	UML Diagrams:
6.1.	Use Case Diagram
Actors:	Customer, Staff, Admin
•	Customer → Book room, Cancel room, Make payment
•	Staff → Check-in, Check-out, Manage room
•	Admin → Manage employees, Generate reports
6.2.	Class Diagram
Class:	Customer, Room, Booking, Payment, Employee
Admin:	
7.	Other Requirements
•	Backup database daily
•	Multi-language support
•	GST/Tax compliance for Billing

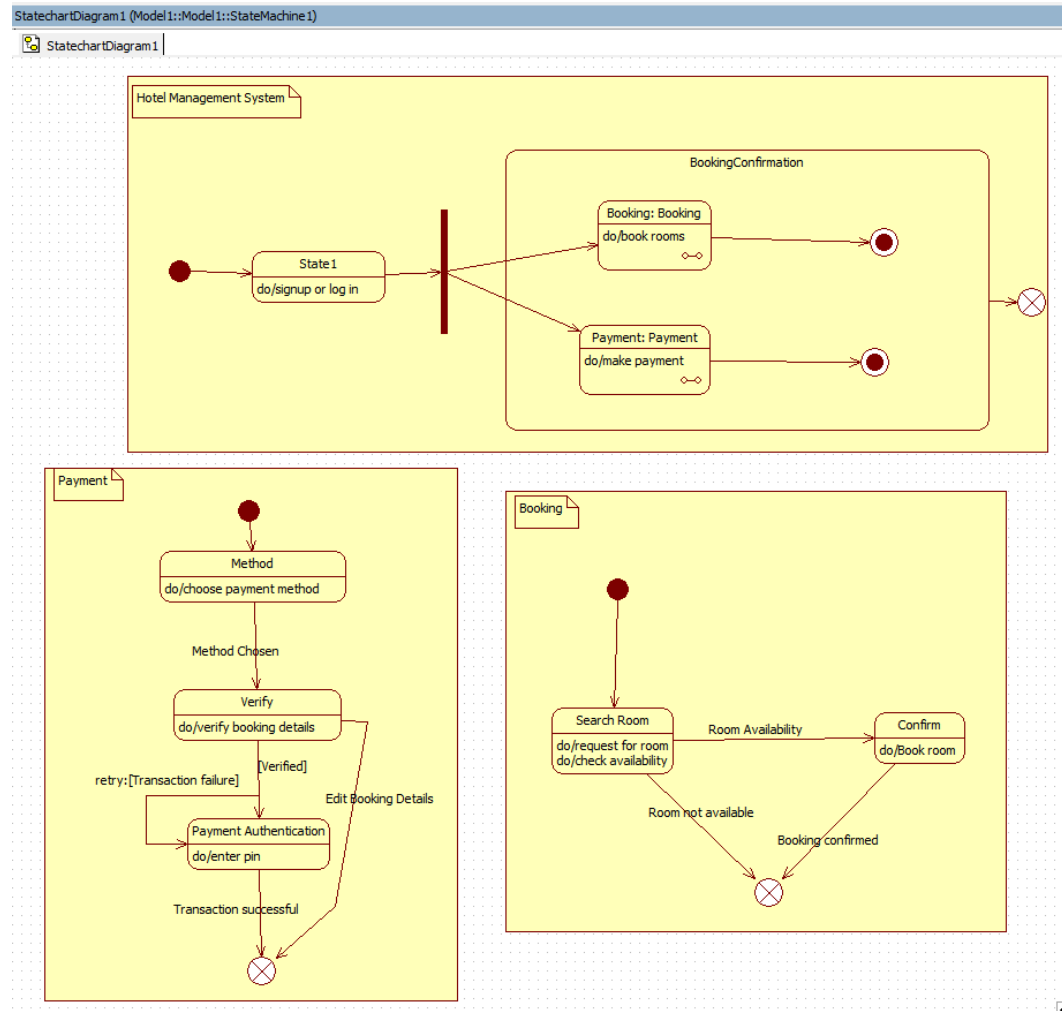
Class Diagram



Explanation:

The class diagram for a Hotel Management System provides a detailed structure of the system by defining core classes—Hotel, Branch, Customer, Room, Employee, Payment, FoodItem, and Stock—alongside their attributes, methods, and intricate relationships. A hotel manages multiple branches, each with its own specifics, while customers can book various types of rooms, such as bedrooms or suites, and make secure payments. Employees are organized by roles like chef, manager, or receptionist, streamlining hotel operations, customer service, and food preparation. Additional components track food items and stock, ensuring an integrated approach to inventory and service management. Enumerations for room types and payment modes help standardize and automate bookings and transactions. By capturing both inheritance (e.g., specialized rooms and employee roles) and composition relationships, the diagram ensures a robust and adaptable design for managing hotel workflows efficiently.

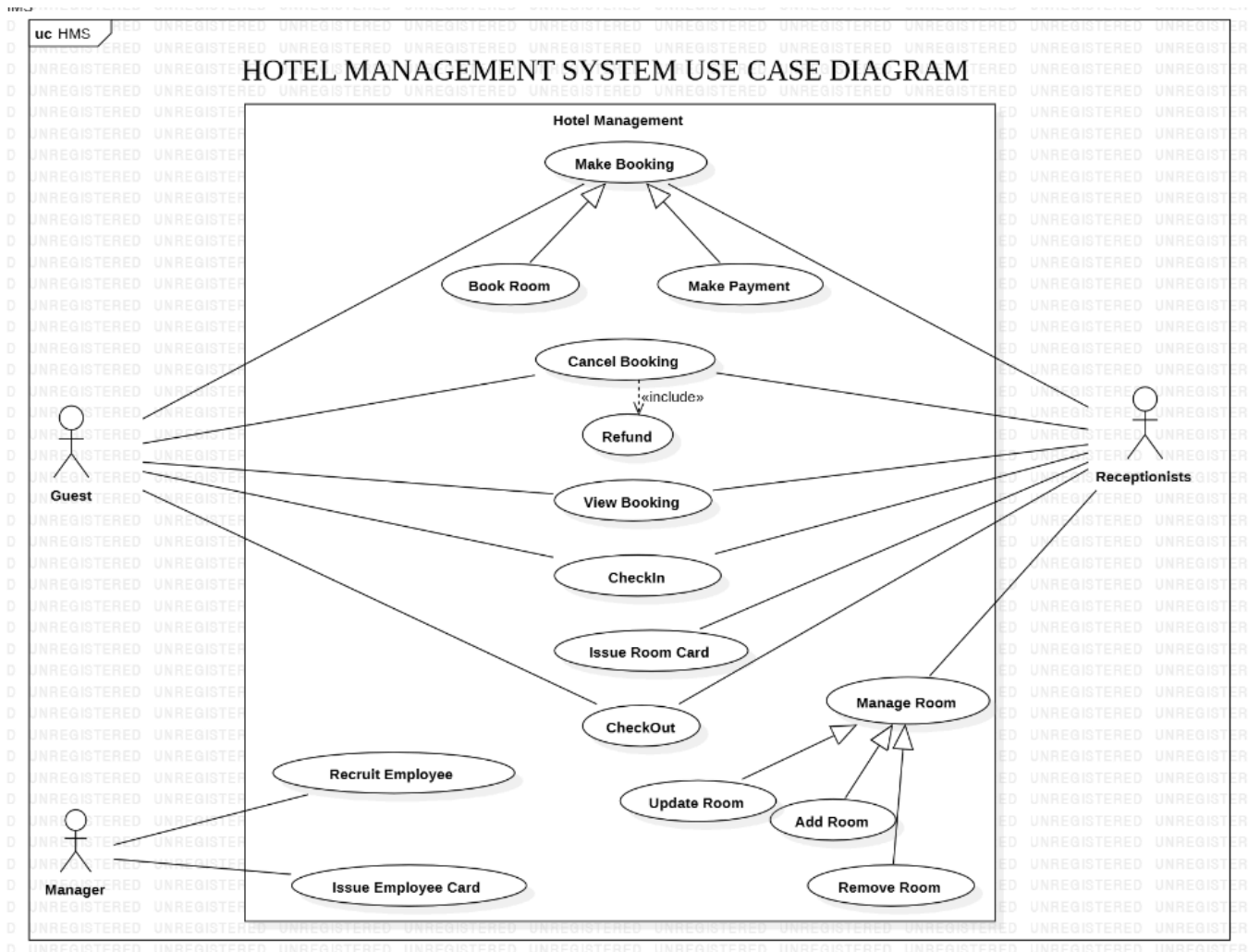
State Diagram



Advanced State Diagram

The advanced state diagram explains the entire hotel room management process. It begins with the system waiting for the user. When a room is booked, the system handles reservation details and then moves to the check-in process, where customer information is recorded and a room is assigned. During the guest's stay, the system enters the Room Occupied state, which includes billing and guest services like food delivery or housekeeping. After the guest checks out, the room goes through cleaning and, if needed, maintenance. Once completed, the room becomes available again. This diagram shows the full cycle of a room—from booking to check-in, stay, check-out, cleaning, and maintenance—providing a deeper view of hotel operations.

Use Case

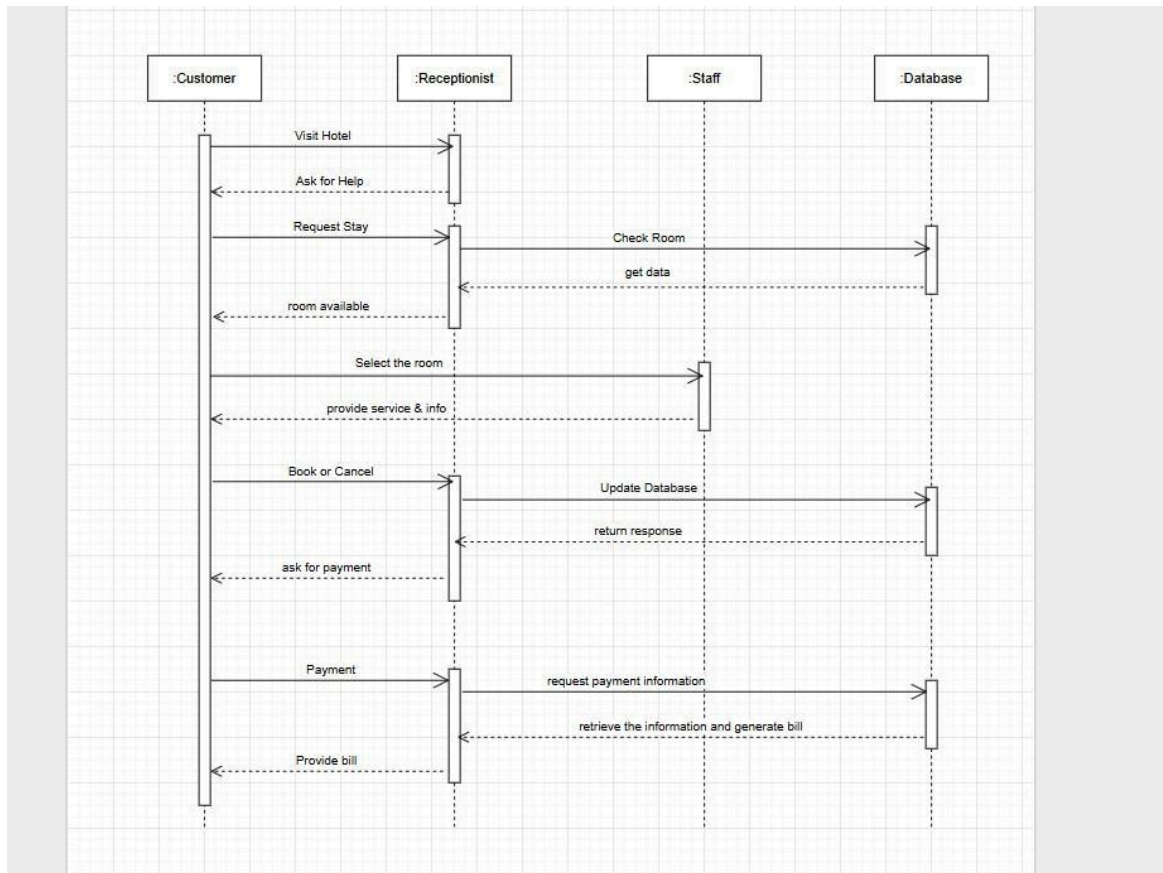


Advanced Use Case Diagram

The advanced use case diagram shows a more detailed view of how hotel staff and users interact with the system. It includes both Admin and User roles along with all supporting processes. The Admin can make reservations, view bookings, handle check-ins and check-outs, calculate bills, and process payments. These actions may involve additional steps such as choosing room types, selecting customers, filtering available rooms, or finalizing a reservation. The User can create customer profiles, check available rooms, view customer lists, and select reservations. These use cases support the admin's work and help the system run smoothly. This diagram gives a clearer picture of the system's internal workflow, showing

how different tasks are connected and how the hotel manages the entire reservation and payment process.

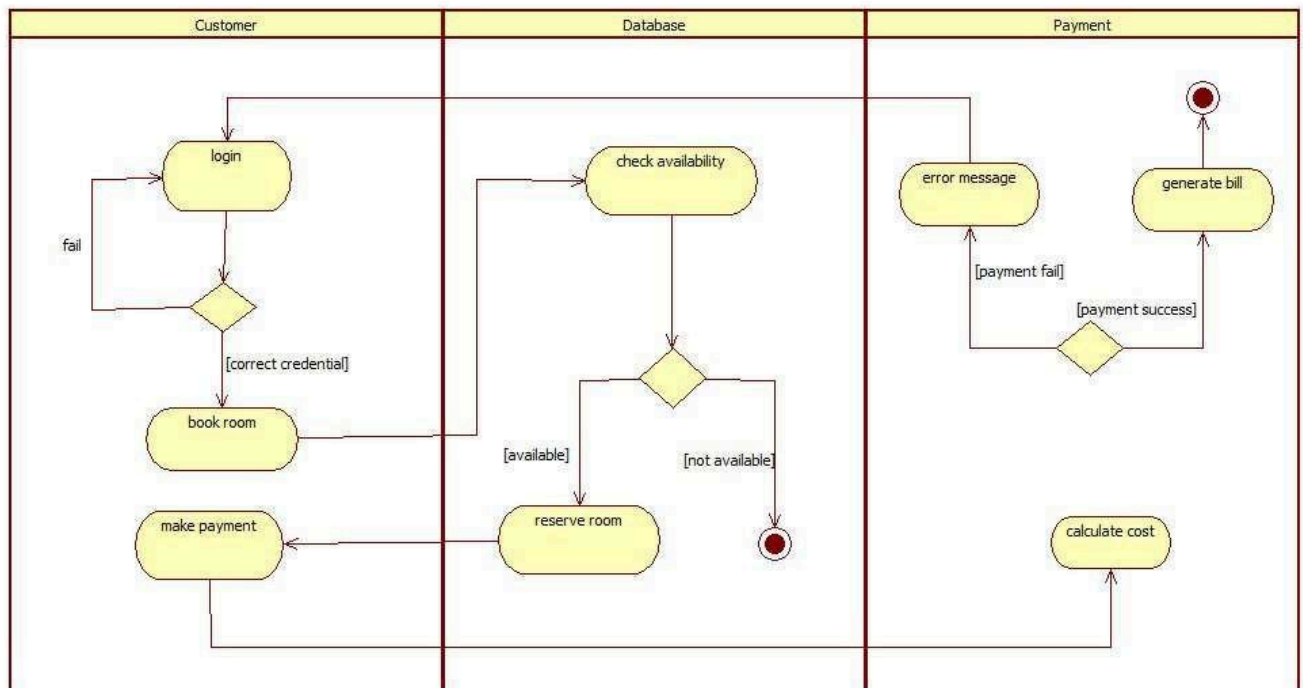
Sequence diagram



Advanced Sequence Diagram (Customer—Receptionist—Staff—Database)

The advanced sequence diagram provides a more complete picture of the hotel booking process. It starts when the Customer approaches the Receptionist for help. The customer requests a room, and the receptionist forwards this request to the Staff, who checks room availability by interacting with the Database. The database returns the room information, and the receptionist shares it with the customer. The customer selects a room, and the receptionist provides the required details or services. If the customer decides to book or cancel, the receptionist updates the Database, which stores the new booking status and returns confirmation. After the booking is confirmed, the receptionist asks the customer for payment. The receptionist retrieves payment details from the database and prepares the final bill for the customer. This advanced diagram shows the full workflow—from request to billing—highlighting the roles of staff and database operations behind the scenes.

Activity Diagram



Advanced Activity Diagram

The advanced activity diagram shows the same process in more detail and separates the actions into three swimlanes: Customer, Database, and Payment.

- In the Customer section, the user logs in, books a room, and makes the payment.
- In the Database section, the system checks if rooms are available and reserves the room when possible.
- In the Payment section, the system calculates the total amount, processes the payment, and generates the bill.

The advanced diagram also includes decision points such as incorrect login, room not available, and payment failure. It presents a clearer and more complete picture of how different parts of the hotel system work together.

2. Credit Card Processing

SRS Document

Date: / /
Page:

LAB 2

CREDIT CARD PROCESSING

1. Introduction

1.1. Purpose:
Purpose of this document is to define the functional & non-functional requirements of credit card processing system (CCPS). System securely authorize, process & manages credit card transactions between customers, merchants & banks.

1.2. Scope:
The system will:

- Accept credit card payments from customers.
- Authorize transactions with issuing banks.
- Provide fraud detection & validation checks.
- Allow merchants to view transaction history & generate reports.
- Ensure compliance with PCI DSS.

1.3. Definitions, Acronyms & Abbreviations

- CCPS - Credit card processing system
- P.C.S - Payment card
- Issuer - The bank that issued credit card
- Acquirer - Bank handling merchant's transaction.

Date: / /
Page:

1.4. References:

- IEEE std 830-1998 - Recommended Practice for Software Requirement Specification

1.5. Overview:
This document specifies system features, constraints, & performance requirements to ensure secure & efficient credit card processing.

2. Overall Description

2.1. Product Perspective:
The CCPS is web based & API driven system that integrates with merchant applications. It communicates with payment gateway & banks to authorize & settle transactions.

2.2. Product Functions

- Card validation
- Transaction authorization
- Fraud detection
- Settlement & fund transfer
- Merchant dashboard & reporting

2.3. User Characteristics:

- Customers: Minimal technical knowledge, primary goal is to make quick payments.
- Merchants: Moderate technical knowledge for monitoring transaction.
- Administrators: Technical staff responsible for monitoring transactions & security.

Date: / /
Page:

2.4. Constraints:

- Must comply with PCI DSS standards
- Must process transactions under 3 seconds

2.5. Assumptions:

- Internet connectivity is required
- Banks provide reliable APIs

3. Specific Requirements:

3.1. Functional Requirements:

- FR1: System shall provide credit card no, expiry & CVV.
- FR2: System shall encrypt all cardholder data using AES-256
- FR3: System shall authenticate customers via OTP
- FR4: System shall process refunds within 48 hours

3.2. Non-functional Requirements:

- Performance: Each transaction must be processed in ≤ 3 seconds.
- Security: Must comply with PCI DSS & meet TLS 1.3 for all communications.
- Scalability: Support at least 1 million transactions per day.
- Reliability: 99.9% uptime.

Date: / /
Page:

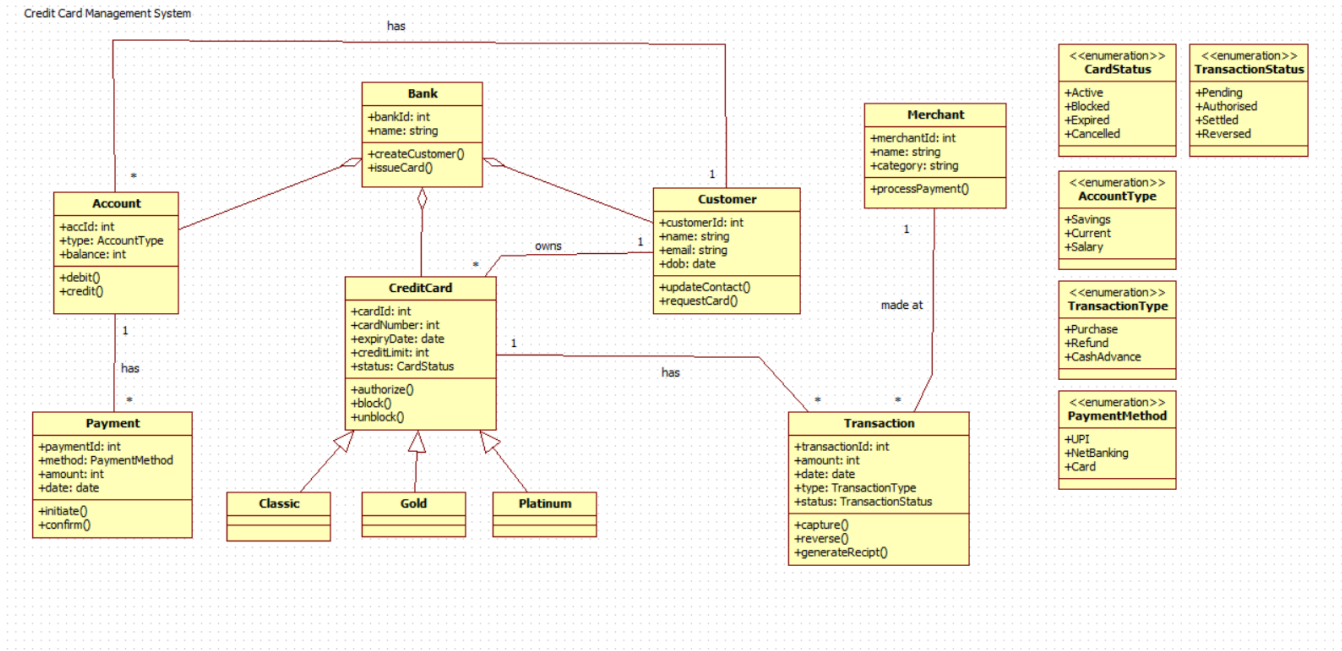
3.3. External Interface Requirements

- User Interface: Responsive web dashboard for merchants & administrators
- API Interface: REST APIs for integration with merchant apps
- Hardware Interface: Secure card reader

4. Appendices:

- Glossary of Terms
- Data flow diagrams
- ER diagram.

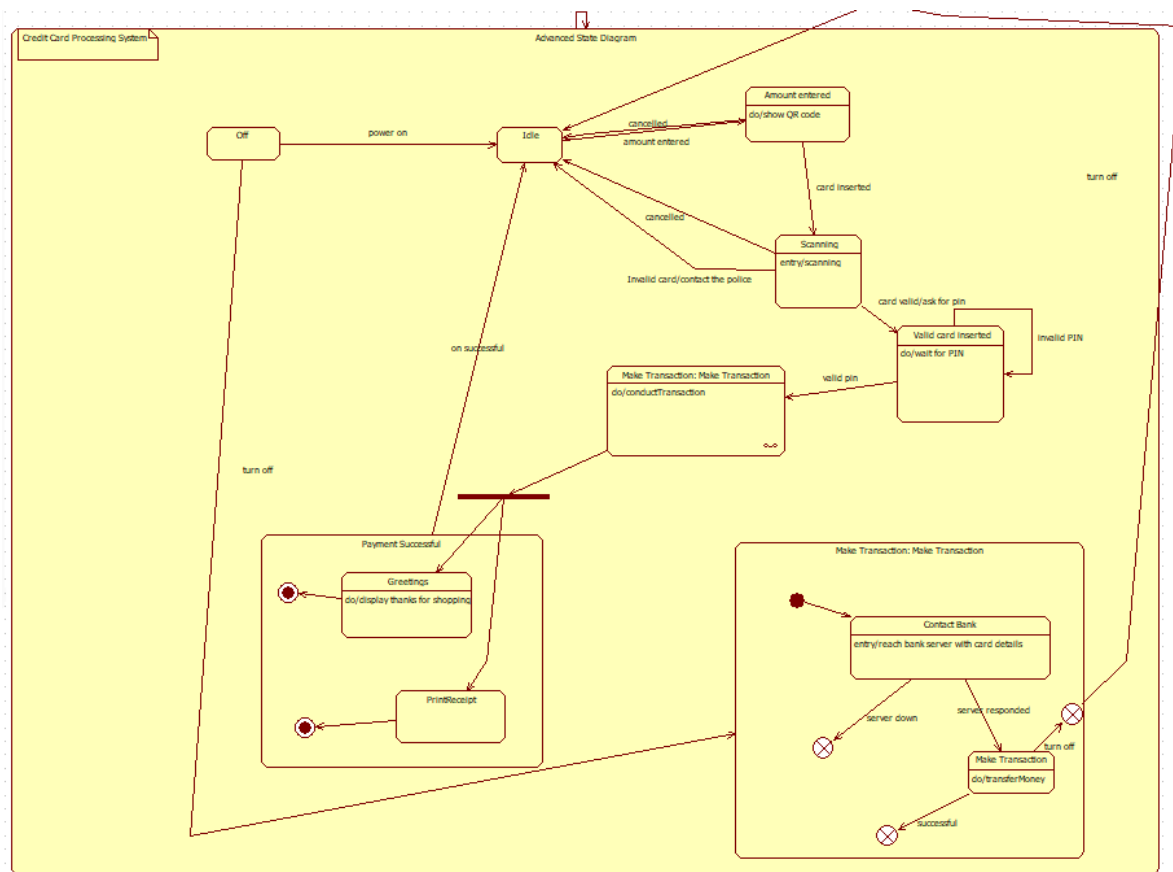
Class Diagram



Explanation

The class diagram represents an online payment processing system that manages customers, merchants, banks, and transactions. The Customer initiates payments using their card, which is linked to a Bank Account. A Transaction is created for each payment, containing details such as amount, date, status, and authorization code. The Bank authorizes and settles payments, while the PaymentGateway facilitates communication between the customer's bank and the merchant. The Merchant receives payments, issues refunds, and handles chargebacks. Additional classes such as Authorization and Refund manage fraud checks, verification, and money returns. Supporting entities like Person, Payment, and the status enumeration define basic attributes and types used throughout the system. Overall, the diagram shows how different components work together to process, validate, and complete electronic payment transactions securely.

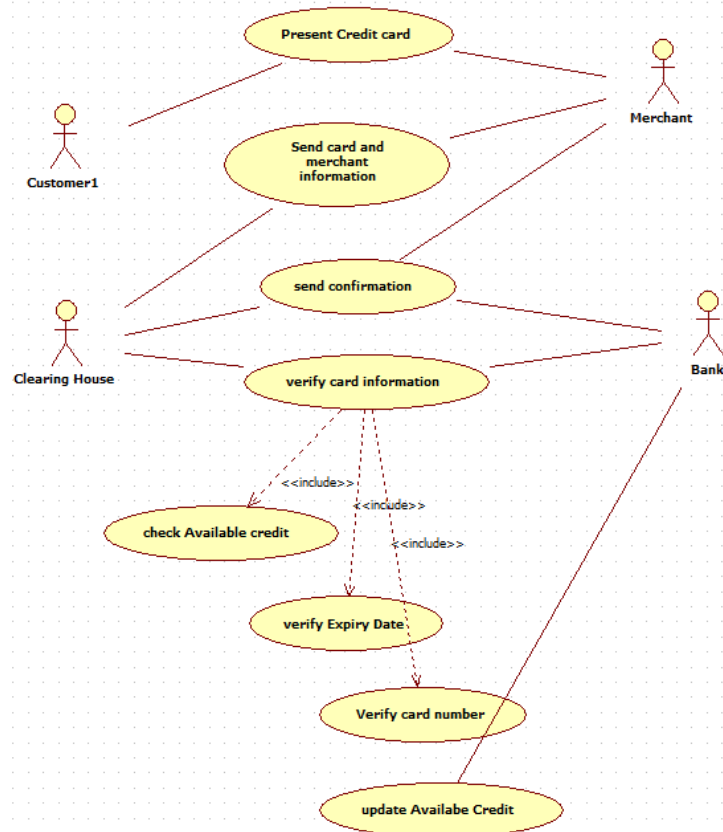
State Diagram



Advanced State Diagram (ATM/Payment Terminal Transaction Flow)

The advanced state diagram describes a more detailed and realistic payment or ATM terminal process. It begins in an Idle state, where the terminal waits for input. A user either enters an amount (for QR payment) or inserts a card. The system validates the card through a scanning process. If the card is valid, the terminal asks for the PIN and moves to the Valid Card Inserted state. After the correct PIN is entered, the process moves to Make Transaction, where the transaction is conducted and sent to the bank for approval. The Control Bank state checks card details and verifies the transaction. If the bank responds successfully, the terminal processes the payment and shows a greeting message, followed by printing a receipt. The diagram also includes multiple exceptional paths such as invalid PIN, invalid card, server down, or cancelled actions. This advanced diagram shows a complete, realistic workflow including scanning, PIN verification, bank communication, success/failure handling, and receipt printing.

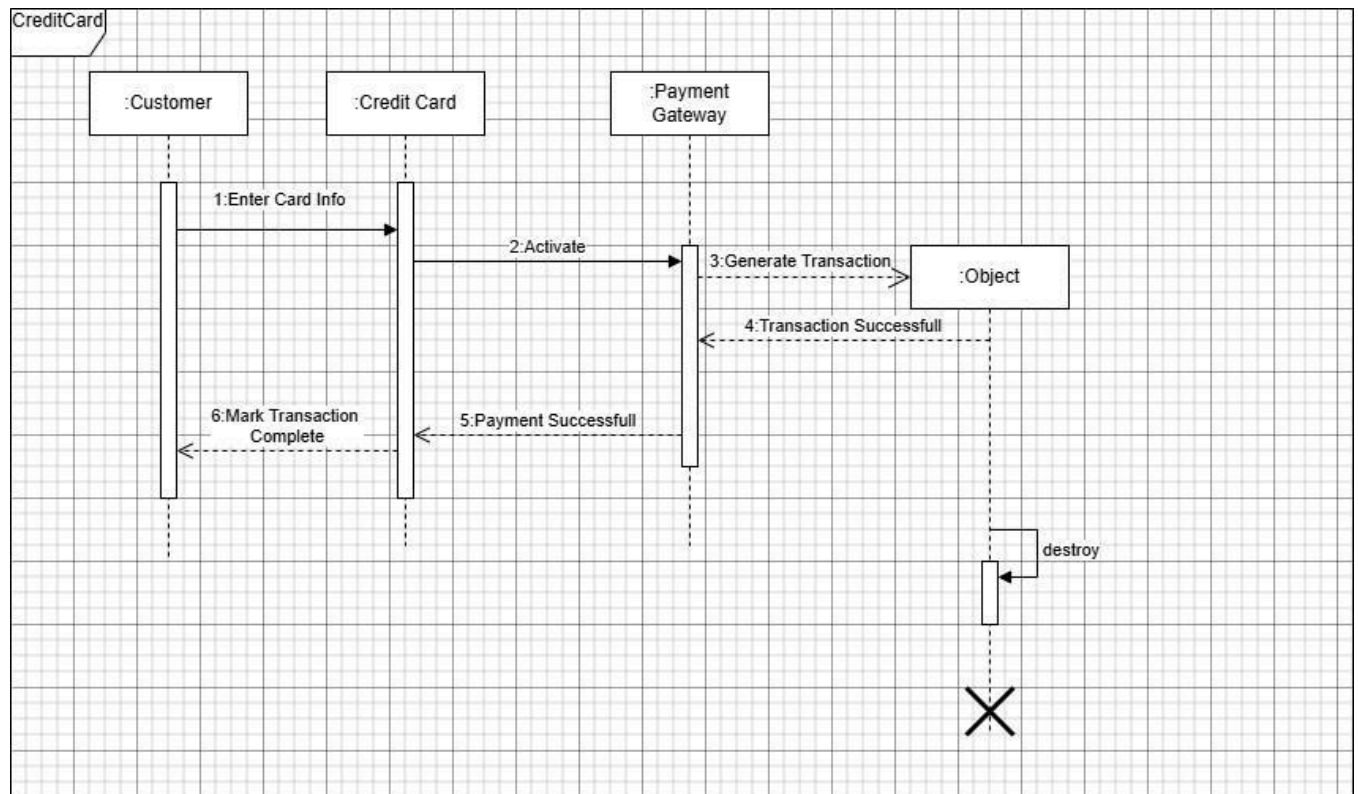
UseCase



Advanced Use Case Diagram (Banking Customer Management System)

The advanced use case diagram presents a much more detailed view of how users and administrators interact with a banking system. Multiple actors participate: the Admin, Customer (User), Bank Server, and Database. The system supports a wider set of processes such as Creating Customer, Assigning Customer ID, Updating Account, Crediting and Debiting Accounts, Viewing Statements and Account History, Transactions, Login/Logout, and secure processes like Encrypting and Decrypting Messages. Many relationships use *include* and *extend*, showing how smaller actions support larger processes. This diagram provides a complete picture of how customer accounts are managed, how transactions flow through the system, and how security is maintained. It shows internal system behavior more clearly than the simple diagram.

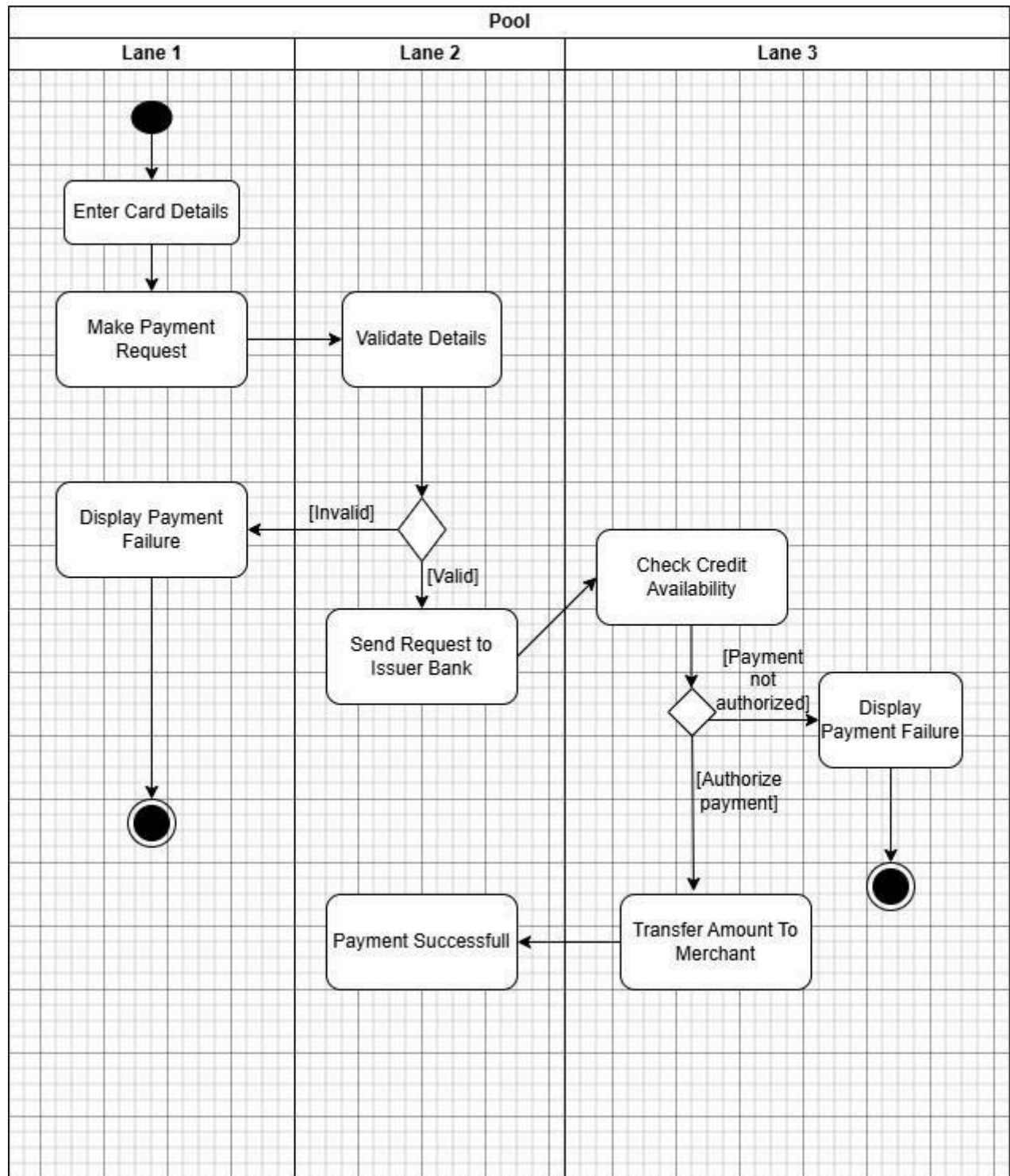
Sequence Diagram



Advanced Sequence Diagram (Customer — Merchant — Bank Server)

The advanced sequence diagram provides a more detailed view of how a credit card payment is handled behind the scenes. It begins when the Customer provides card details to the Merchant for payment. The merchant then sends an authorization request to the Bank Server. The bank server validates the card information, checks the account status, and approves or denies the transaction. If approved, the bank sends back a confirmation to the merchant, who then informs the customer that the payment was successful. Finally, the merchant updates the customer's statement or transaction history. This advanced diagram shows a more complete workflow, including authorization, validation, approval, and transaction recording.

Activity Diagram



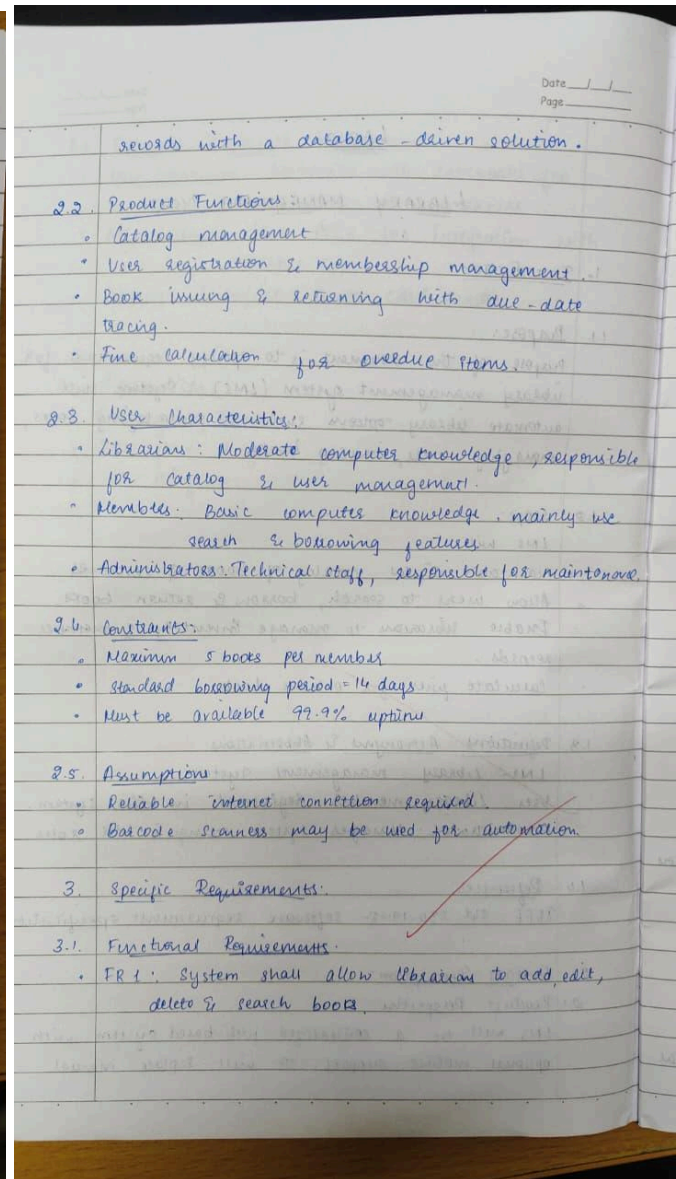
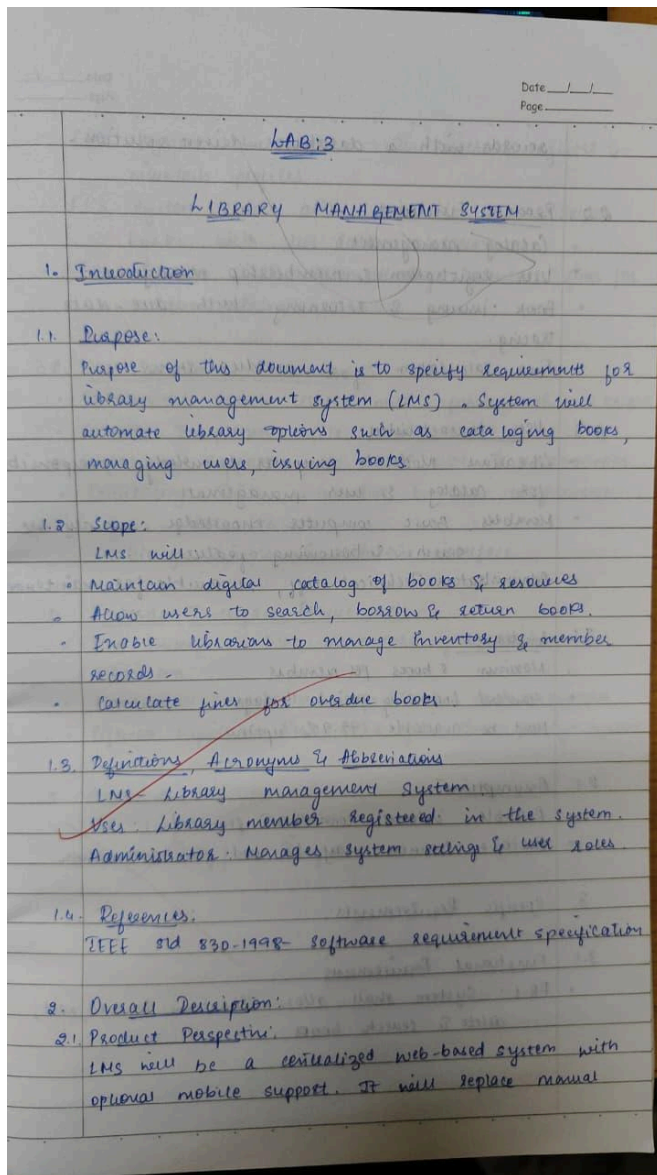
Advanced Activity Diagram (Credit Card Application & Processing System)

The advanced activity diagram presents a more detailed workflow of how a credit card application is handled. It begins when an applicant makes an inquiry about credit cards. The credit card processing system responds by displaying available card options. The applicant then selects a card and decides whether to proceed. If they continue, the system shows the required documents and information needed for application. The applicant gathers and submits these requirements. The system then validates the submitted information, checking eligibility criteria. Once everything is confirmed, the system displays the approved or completed application. This diagram covers more steps and interactions compared to the simple one and offers a clearer view of how credit card applications are processed from inquiry to validation.



3. Library Management System

SRS Document



- FR2: System shall allow new users to register & maintain profiles.
- FR3: System shall allow users to issue & return books with due date validation.
- FR4: System shall automatically calculate fines for overdue returns.

3.2. Non-Functional Requirements

- Performance: Search results must appear within 2 seconds.
- Security: Implement role based access.
- Scalability: Support upto 10,000 users & 100,000 books.
- Reliability: Daily automated backups to prevent data loss.
- Usability: Simple & intuitive interface.

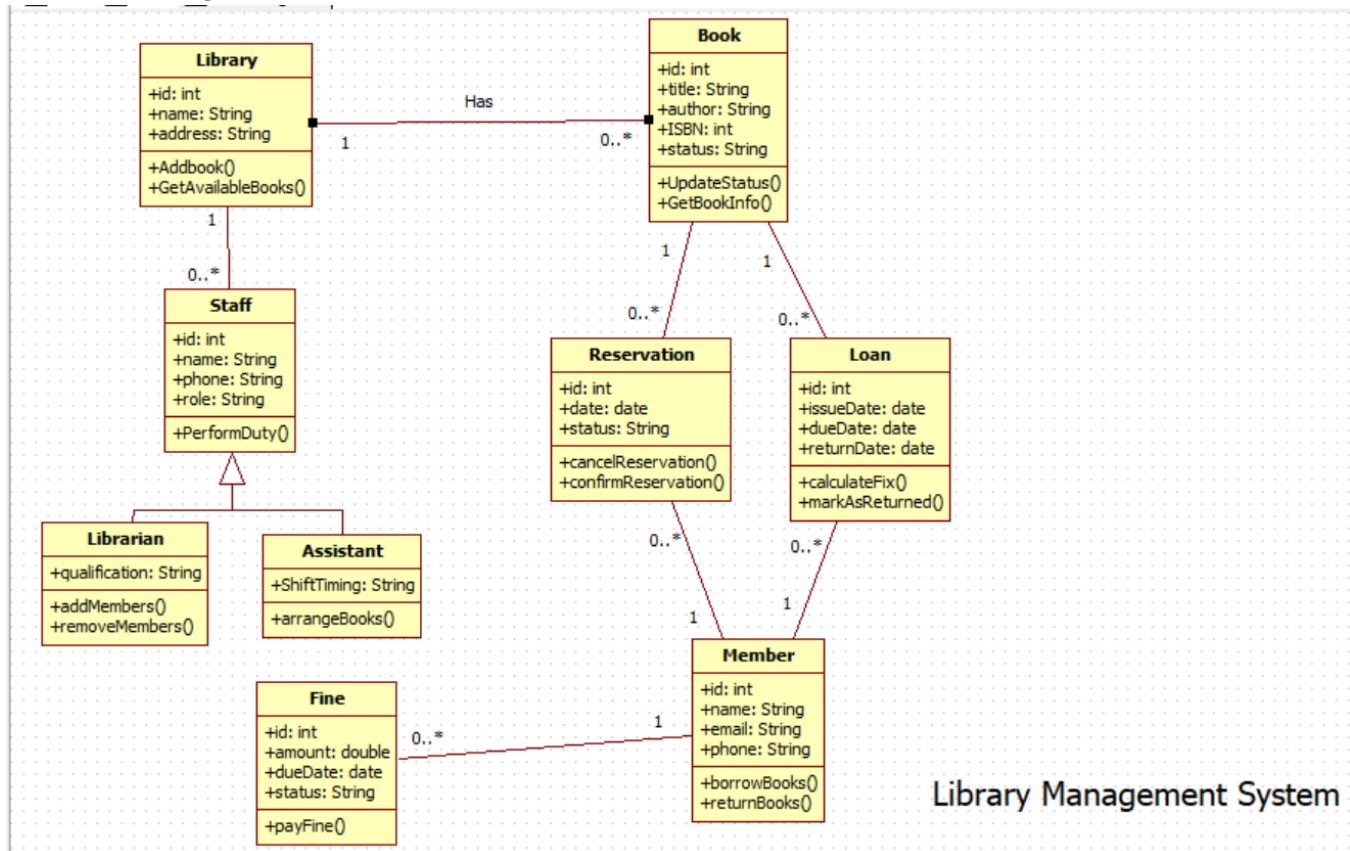
4. External Interface Requirements

- User Interface: Web dashboard for members & librarians.
- Hardware interface: Barcode/RFID scanner support.
- Software support: Rest APIs for third party system.

5. Appendix

ER diagram & Database
Sample Data flow Diagram.

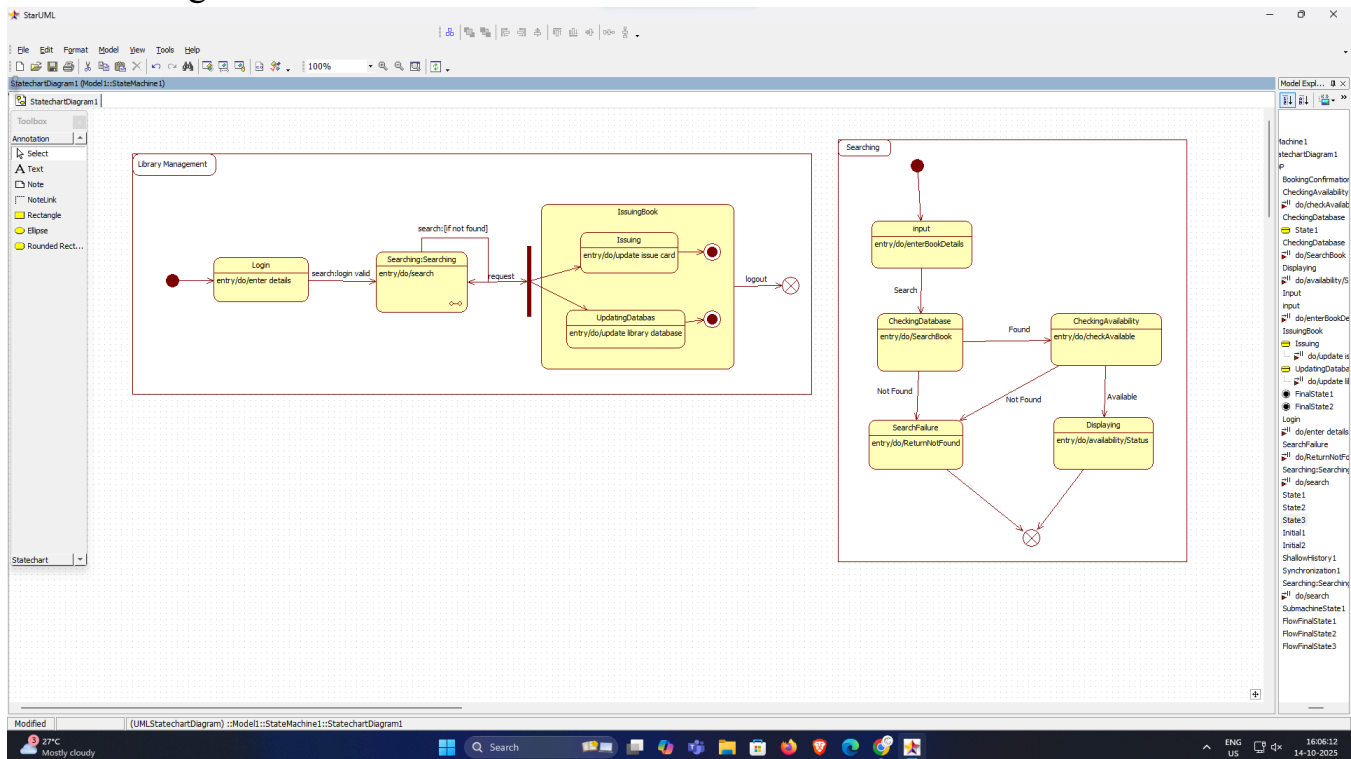
Class Diagram



The class diagram represents a Library Management System where books, members, librarians, and transactions interact to manage library operations. The Book class stores details such as author, edition, price, and purchase date, and it is specialized into different types like Journals, Magazines, and Subject Text through inheritance. Members of the library—categorized as Students and Faculty—send book requests and borrow books. The Librarian plays a central role by searching books, verifying members, issuing books, calculating fines, and handling book returns. The librarian also updates the status of books and ensures that library policies like issuing limits are followed.

The system also keeps track of borrowing activities using the Transaction class, which stores the transaction ID, book ID, issue date, and due date. When members return books late, the Fine class is used to generate bills, update fines, and store payment information. Members pay these fines, and the librarian or system records them for future reference. Overall, the class diagram shows how different components—books, members, librarians, transactions, and fines—work together to ensure smooth operation of the library.

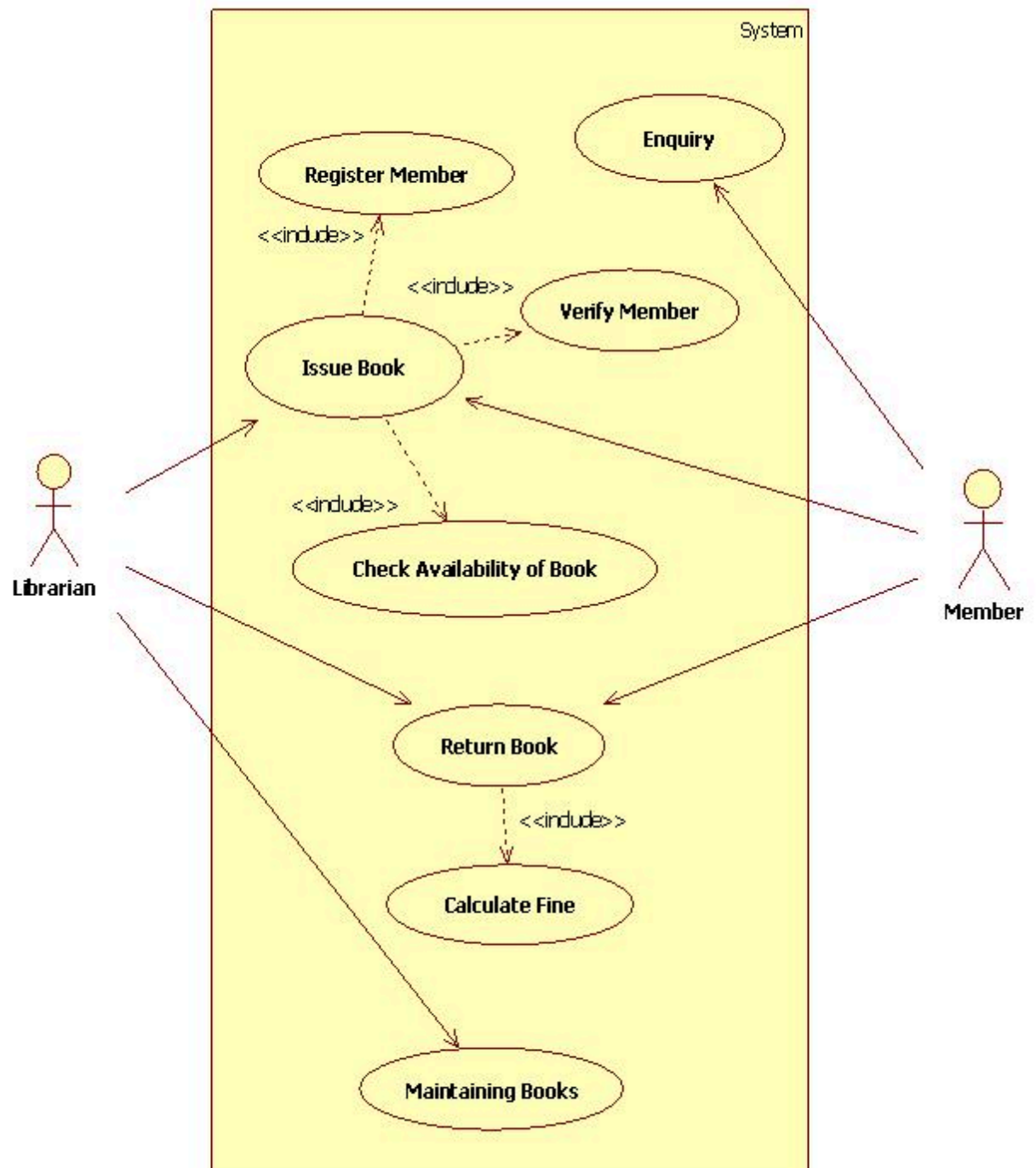
State Diagram



Advanced State Diagram

The advanced state diagram provides a more complete picture of the library's book issuing and returning process. It starts with the user searching for a book. If the book is available, the system verifies stock and moves to the issue state, where member and library records are updated. If the book is unavailable, the system places the user in a waitlist and notifies them when the book becomes available. Once issued, the system moves into the Borrowed state, where the due date is monitored. From here, the user may renew the book or return it. Upon returning, the system checks for damage or late return and computes fines if applicable. If a fine exists, the user proceeds to Fine Payment; otherwise, the process ends. This diagram captures the full lifecycle of a borrowed book—from search and waitlist to issuing, borrowing, returning, renewal, and fine handling.

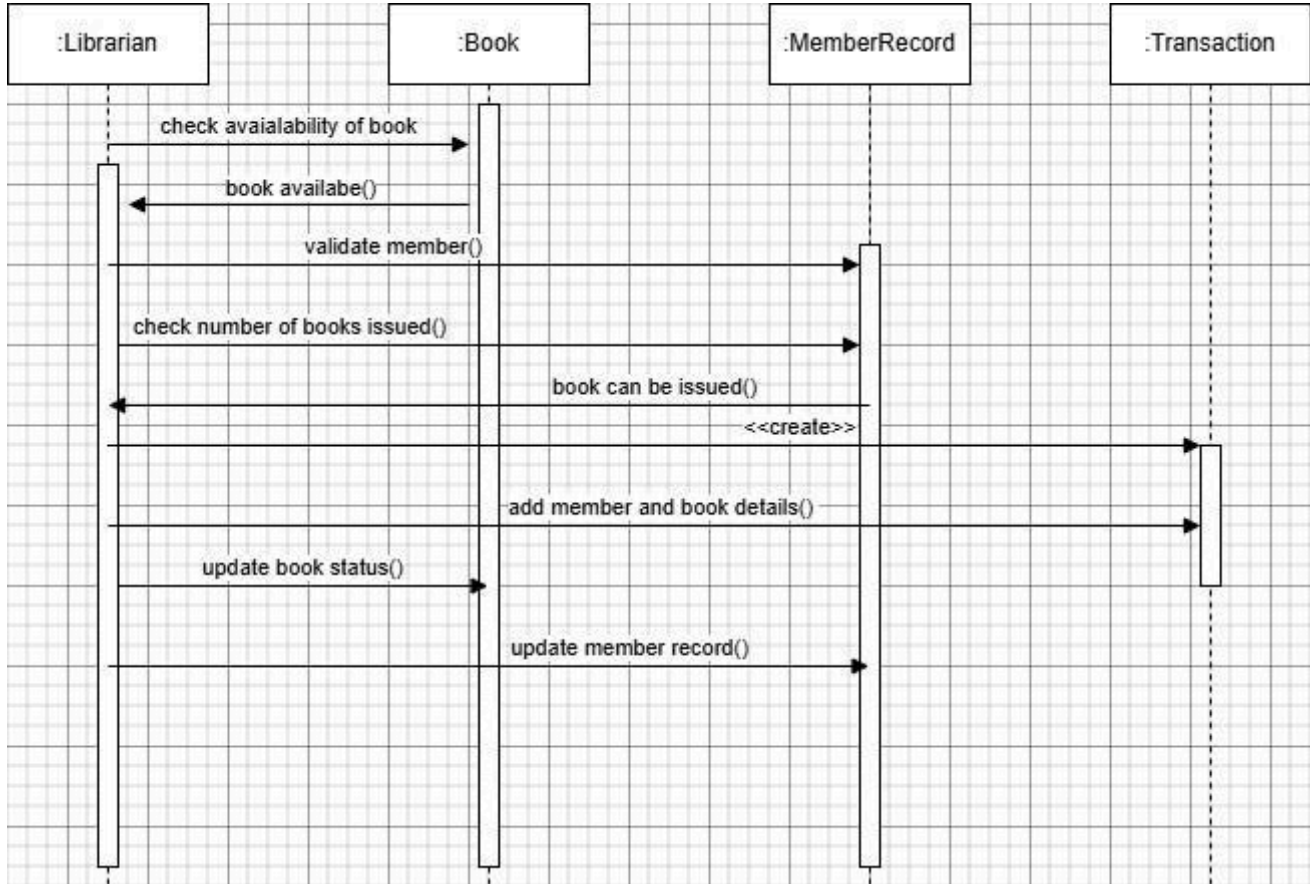
UseCase



Advanced Use Case Diagram

The advanced use case diagram presents a more detailed and comprehensive view of the library system by involving multiple actors—User (Student/Staff), Libraries, and the Library Database. It includes a wider range of use cases such as Authentication, Requesting New Books, Reserving Books, Renewing Books, Paying Fines, Providing Feedback, Registering New Users, and Filling Forms. The system also manages book records using operations like Adding, Updating, and Deleting Records, as well as preparing the library database. Several use cases use *include* and *extend* relationships to show dependencies and optional behaviors such as invalid login, invalid ID, or invalid renewal. This advanced diagram gives a full picture of how users interact with the system, how library data is maintained, and how operations extend beyond simple issuing and returning of books.

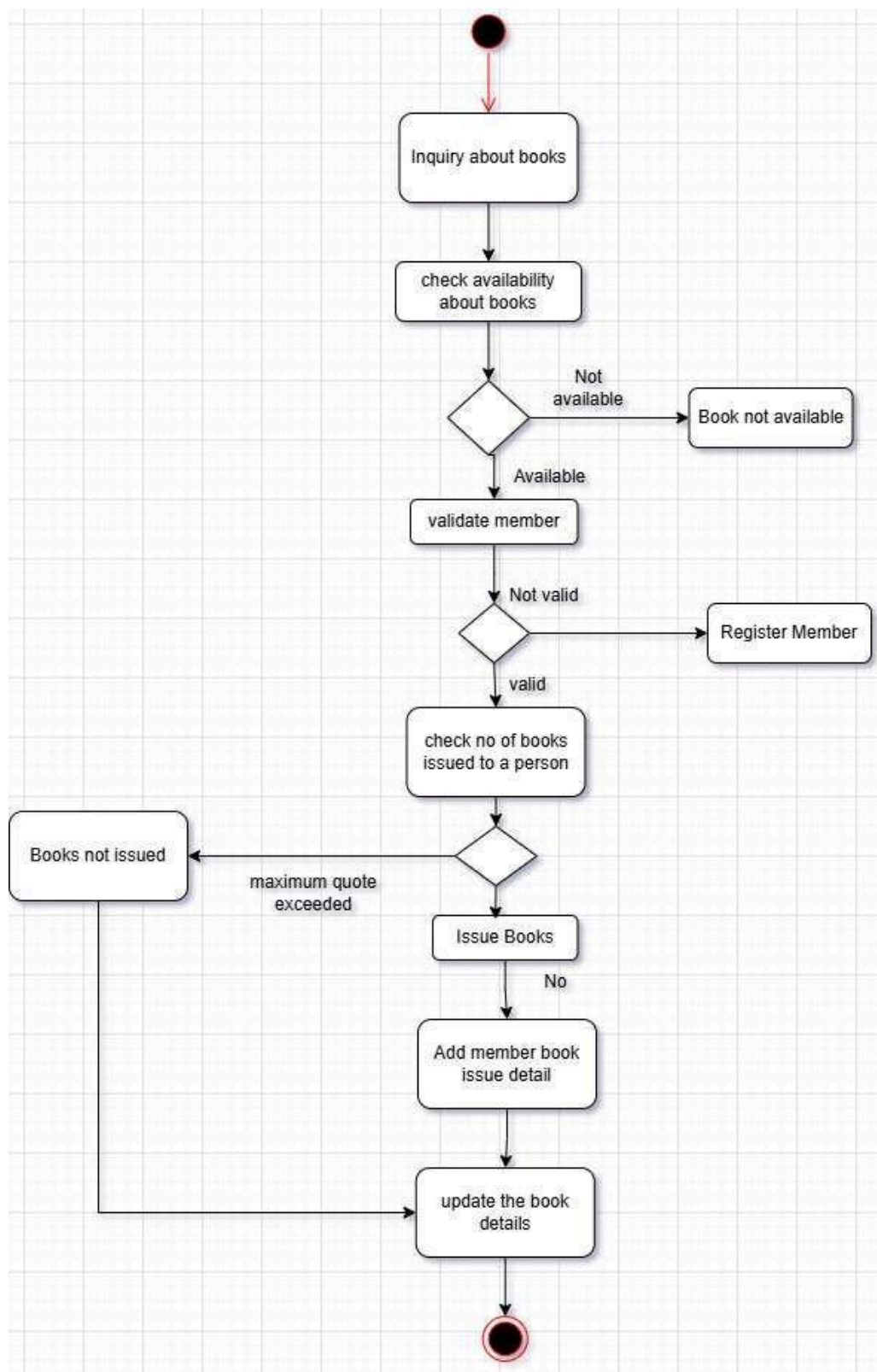
Sequence Diagram(Advanced)



Advanced Sequence Diagram (Librarian — Book — MemberRecord — Transaction)

The advanced sequence diagram provides a more detailed view of the book-issuing process. It starts when the Librarian checks the availability of a selected book. The Book object responds with availability status. The librarian then validates the member and checks how many books the member has already borrowed. If issuing is allowed, a Transaction record is created, containing details of the member and the book. The librarian then updates the book status, marking it as issued, and updates the member's record to reflect the new loan. This detailed diagram shows all internal checks—availability, member validation, issuing limits—and the creation of proper records, giving a complete view of how the system handles book issuing behind the scenes.

Activity Diagram(Advanced)



Advanced Activity Diagram

The advanced activity diagram provides a more detailed and complete view of the book-issuing workflow. In addition to checking availability and validating the member, it clearly shows decision points such as book unavailability, invalid member status, and quota limits. It also includes additional actions like adding member book issue details, updating the complete book record, and handling both “Book not issued” and “Issue Books” outcomes. By showing all alternative paths and internal updates, the advanced diagram presents the full operational logic of how the library processes book inquiries, member eligibility, quota checks, issuing actions, and record maintenance. It reflects the complete backend process of issuing a book in a real library system.

4. Stock Maintenance System

SRS Document:

Date: / /
Page:

Stock Maintenance System

- Introduction
 - Purpose:

Purpose of this document is to define requirements of stock maintenance system (SMS). System manages inventory levels, records stock transaction & generates reports to ensure efficient monitoring of goods.
 - Scope:

System will:

 - Maintain a digital record of all stock items.
 - Track incoming & outgoing stock.
 - Alert users when stock falls below minimum level.
 - Provide reports on current stock, shortages & transaction history.
 - Definitions, Acronyms, Abbreviations:

SMS - Stock Maintenance System
Stock item - An individual product or material maintained in the inventory.
 - References:
 - IEEE std 820-1998 - Software Requirements specification
- Overall Description:
 - Product Perspective:

The stock maintenance system is a standalone application with a database backend. It interacts with users through a web interface. It replaces manual stock records with a digital

Date: / /
Page:

Solution & can integrate with accounting.

- Product Function

Key functions include:

 - Stock entry & modification
 - Real-time inventory tracker
 - Automatic alerts for low stock
 - Search & filter items by category, supplier or date
 - Report generation (PDF)
 - Secure login & user roles
- User Characteristics
 - Admin: Technical knowledge, manages system & user roles
 - Staff/User: Basic computer knowledge, manages stock operation
- Constraints:
 - System must run on windows/linux with internet access
 - Database: MySQL/PostgreSQL
 - Programming language: Java/Python/Node.js
- Assumptions & Dependencies:
 - Users have stable & network connectivity.
 - Users maintain valid login credentials.
- Specific Requirements
 - Functional Requirements:
 - User Management:

FR1: Admin can create, update, & delete user

account.

FR2: System must authenticate users before access.

2. Stock Management

FR3: Add new stock with details

FR4: Update stock when items are sold or purchased.

FR5: Delete stock entries if discontinued.

3.2.3 Non-Functional Requirements

- Performance: Support at least 100 concurrent users.
- Usability: Simple & intuitive UI.
- Security: Encrypted passwords, role-based access.
- Reliability: Expandable for multiple warehouses.

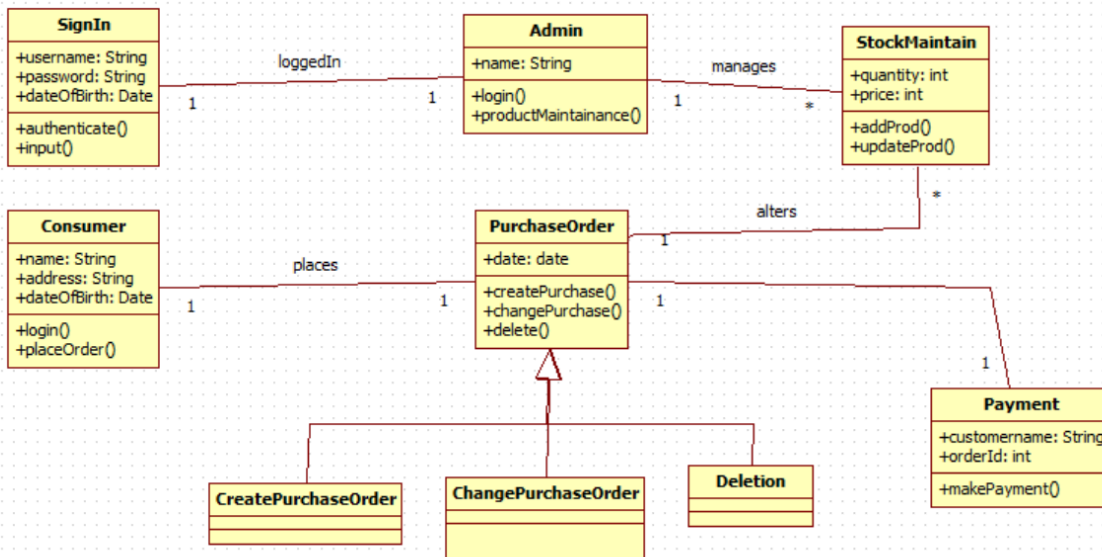
A. System Features

- Admin features: Manage users, categories & reports.
- Staff features: Manage stock, update inventory & view reports.
- Reports: Daily/weekly/monthly summaries with export option.

B. Other Requirements

- Data backup & recovery must be supported.
- Must comply with organisational IT practices.
- Future upgrade for barcode.

Class Diagram

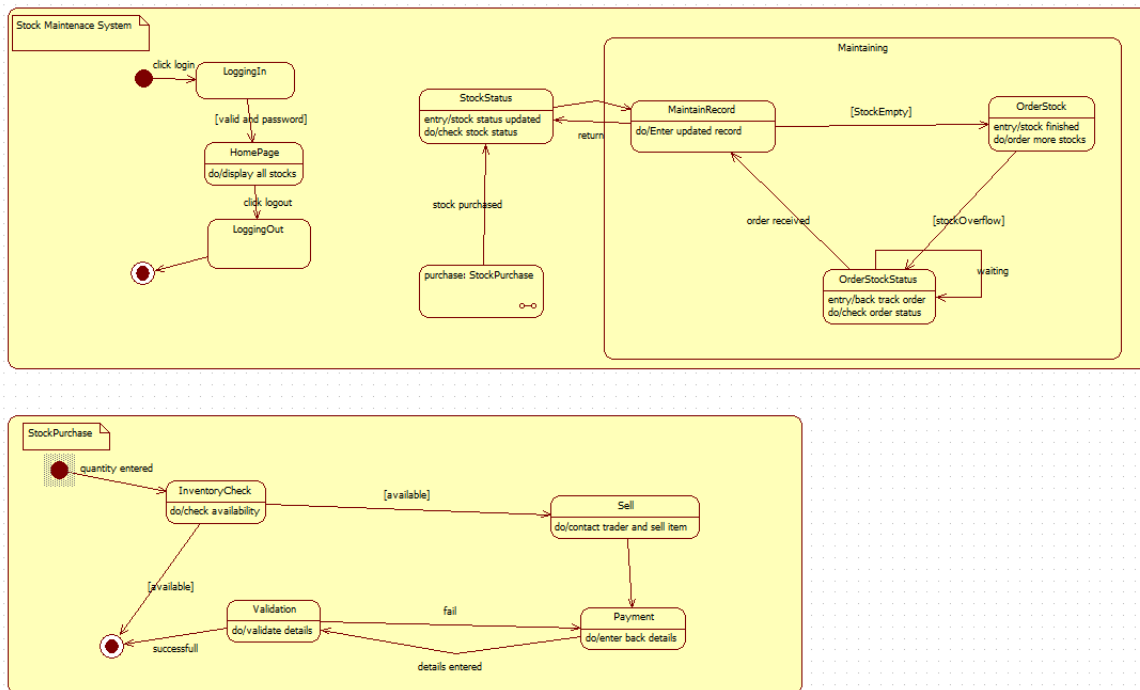


Stock Management System

Explanation:

This class diagram visually represents the key entities and their relationships in a Sales Management System, showing how different objects such as Users, Customers, Admins, Suppliers, Stocks, Products, Transactions, and Sales interact within the system. Each class is defined with specific attributes and methods, and the arrows indicate relationships—such as Customers making Transactions, Admins managing Stocks and contacting Suppliers, and Sales being associated with Customers—highlighting the flow of information and control throughout the platform. This structure helps ensure efficient handling of sales, inventory, user actions, and supplier coordination, forming a cohesive framework for managing business operations digitally.

Advanced State Diagram

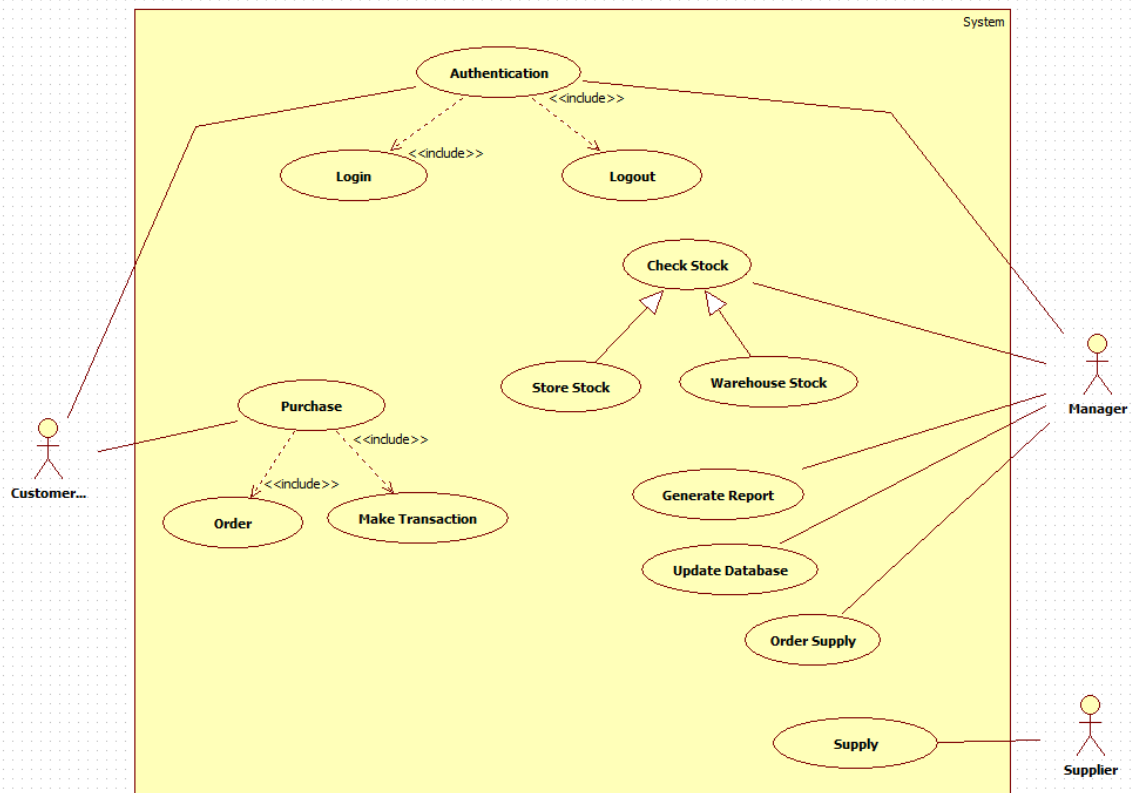


Advanced State Diagram:

This advanced state diagram models the Stock Maintenance System, focusing on two core processes: managing overall stock status and handling stock purchase transactions. The top section illustrates user actions like logging in, navigating the homepage to view stocks, initiating stock purchases, and logging out. Once inside the maintenance area, states transition between updating records, ordering new stock if inventory runs out, and tracking the status of those orders—ensuring stock levels remain sufficient through cyclic monitoring and updating.

The lower section details the Stock Purchase process, starting when a quantity is entered. The flow continues with an inventory check; if stock is available, the item is sold and the system validates and processes the payment. If any details are invalid, the process prompts re-entry of information. Successful transactions complete the state flow, while unavailability or validation failure leads to process termination. Overall, the diagram effectively captures the operational logic and interdependencies involved in maintaining stock and processing purchases within an inventory system.

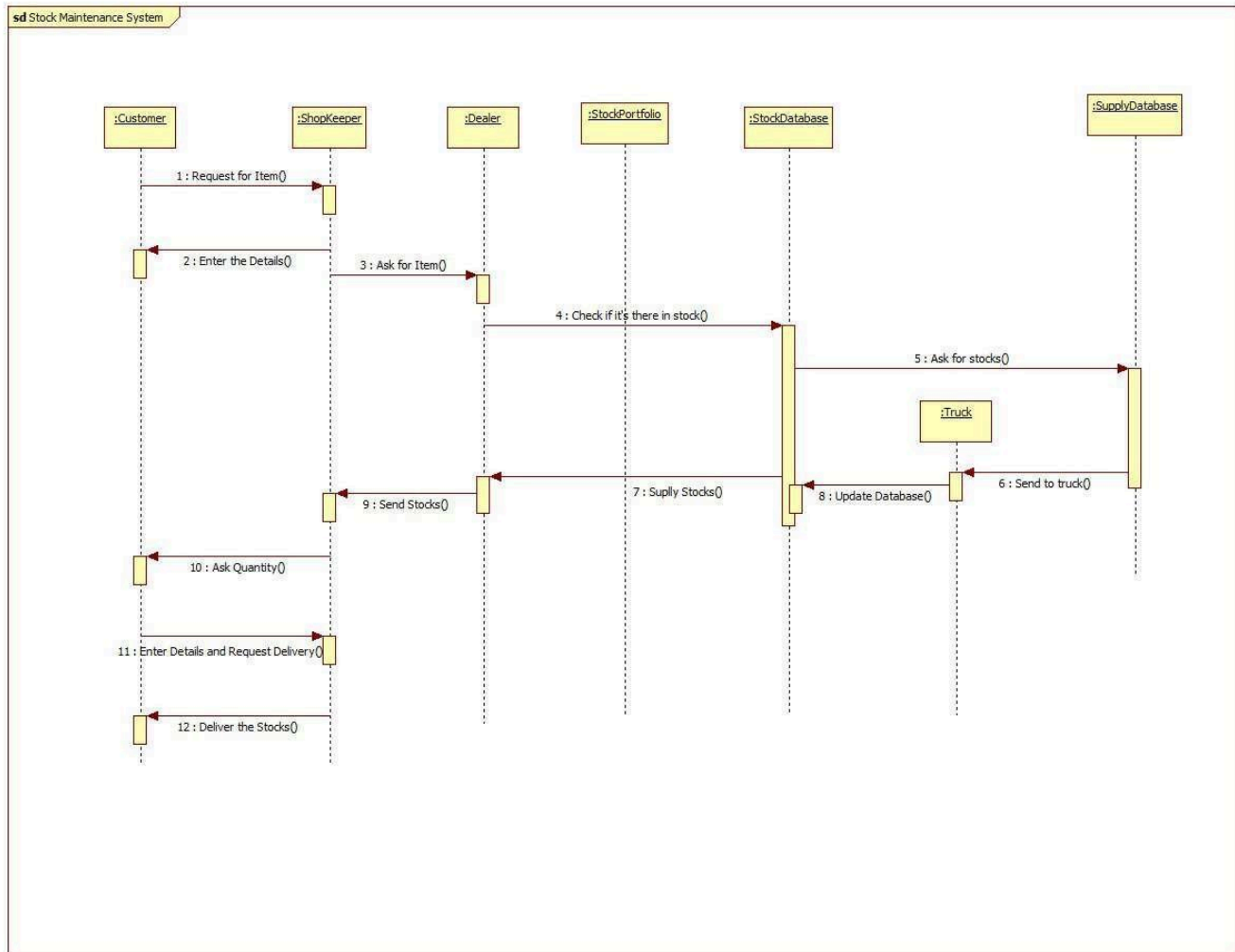
Advanced Use Case:



Advanced Use Case:

This use case diagram outlines the roles of Manager, Store Staff, and Supplier in a retail system. The Manager oversees buying stock, making payments, and supervising staff. Store Staff handle inventory reporting, product quality checks, and selling stock. The Supplier delivers orders, receives payments, and manages quality issue reports. Relationships like $\diamond$ and $\diamond$ show dependencies—for example, buying stock includes giving payment and may extend to reporting quality issues, which in turn includes returning damaged goods. This structured view clarifies responsibilities, enhances system design, and ensures smooth coordination among stakeholders for efficient inventory and supply chain management.

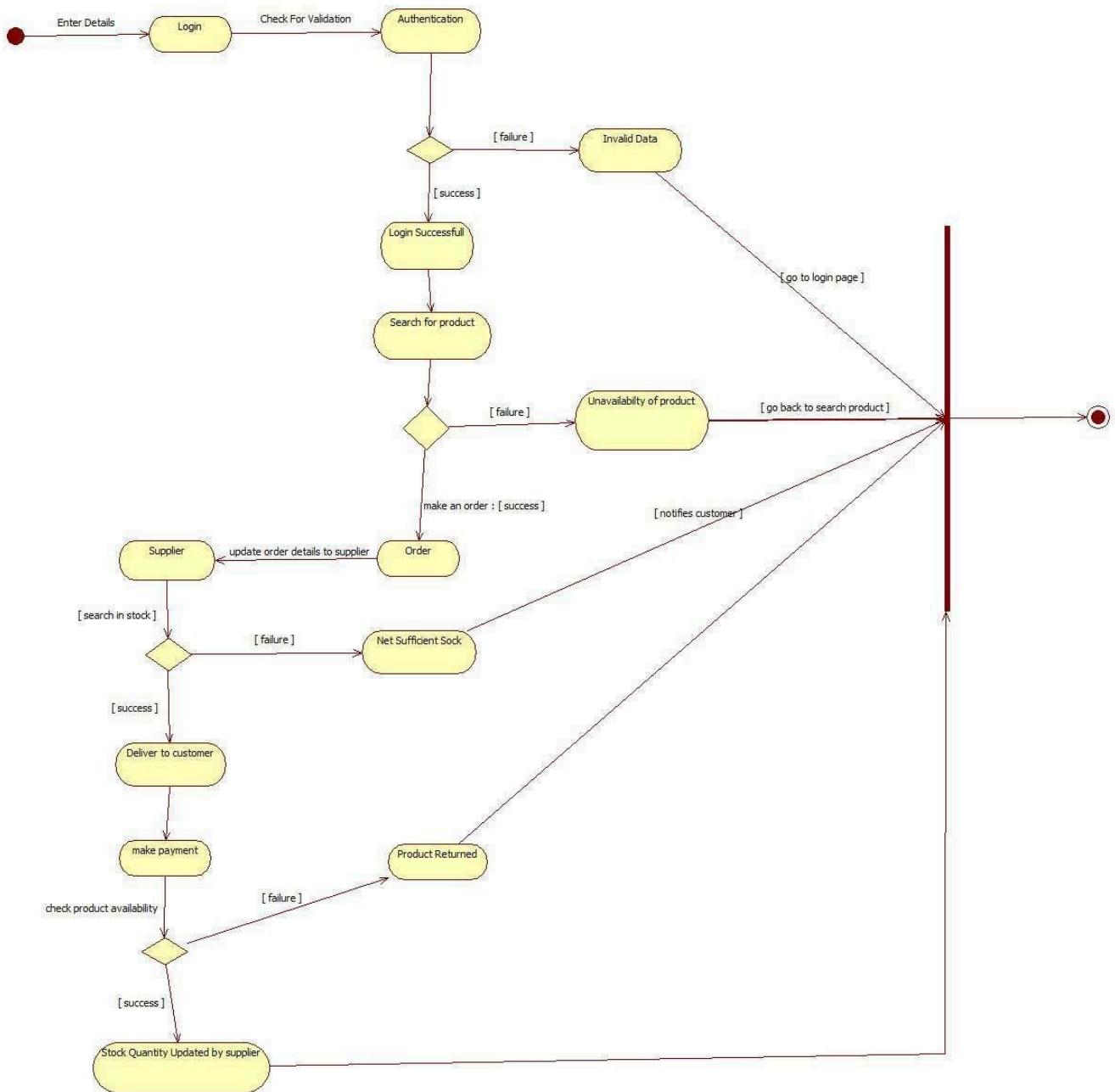
Advanced Sequence Diagram:



Advanced Sequence Diagram:

This UML sequence diagram for the Old Stock Management System outlines the step-by-step interaction between entities like Customer, Login, Authentication, Search Product, Order, Supplier, and Stock. The process begins with customer detail entry and validation. Upon successful login, the customer searches for a product and checks its quantity. If unavailable, the system prompts them to search for alternatives. Once an order is placed, the supplier is notified, and delivery is arranged. The customer pays, stock quantities are updated, and any quality issues can be reported, triggering a refund process. This flow ensures efficient order handling, inventory tracking, and customer satisfaction.

Advanced Activity Diagram:



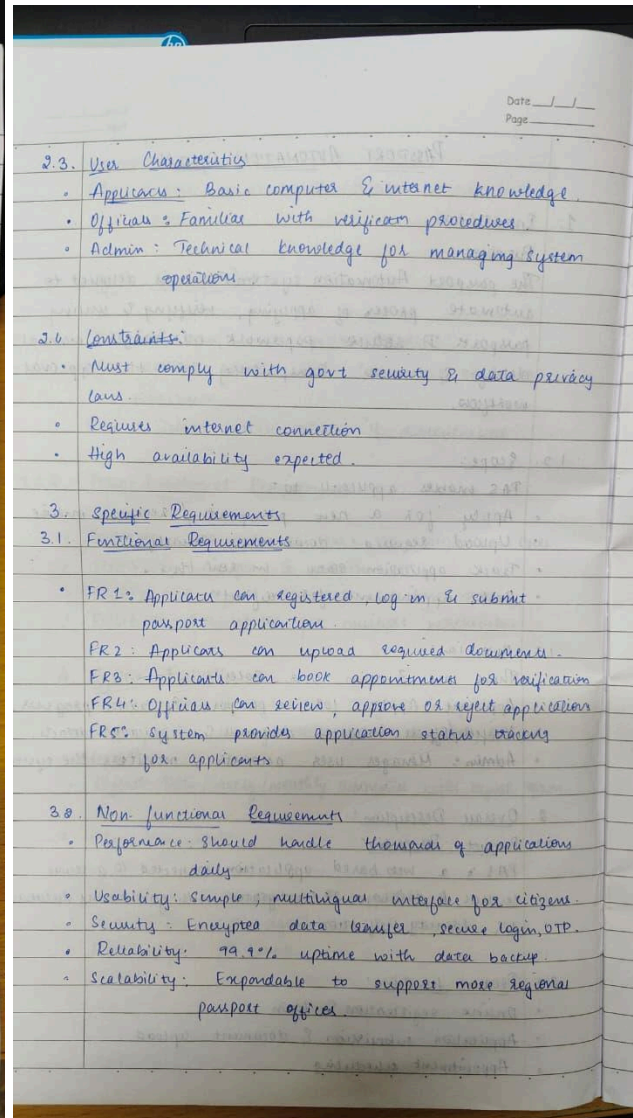
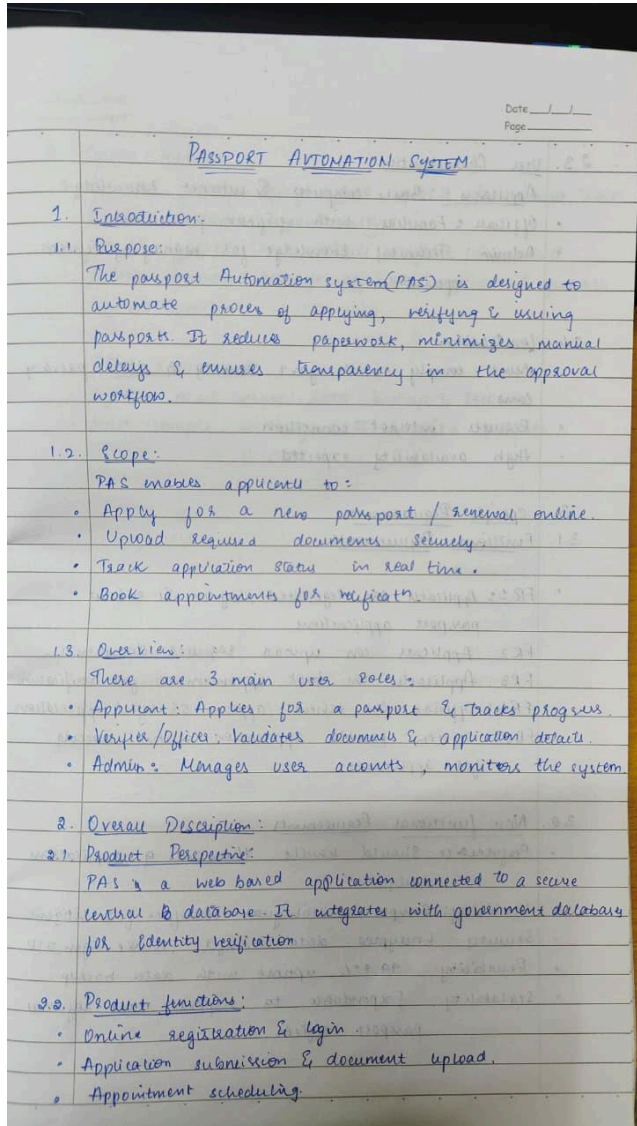
Advanced Activity:

After the order is successfully placed, the system interacts with the supplier to fetch stock and order details. The supplier searches inventory and updates the system accordingly. If delivery fails, the customer is notified immediately. Successful deliveries lead to the payment phase, where transaction failures are also communicated. This decision-based flow ensures transparency and error handling at each step. The supplier plays a critical role in maintaining stock accuracy and fulfilling orders. The system's ability to notify users at every failure point enhances customer experience and operational reliability, making the process robust and user-friendly.

The final steps involve quality checks and stock updates. Once the product is delivered, the customer can report any quality issues. If a defect is found, the system initiates a refund process. Meanwhile, the supplier updates stock quantities to reflect the latest inventory status. This ensures real-time tracking and accountability. The flowchart's design emphasizes decision points and feedback loops, allowing for efficient handling of exceptions. By integrating supplier coordination, customer notifications, and inventory updates, the system maintains a seamless shopping experience while ensuring operational integrity and customer satisfaction.

5. Passport Automation System

SRS Document:



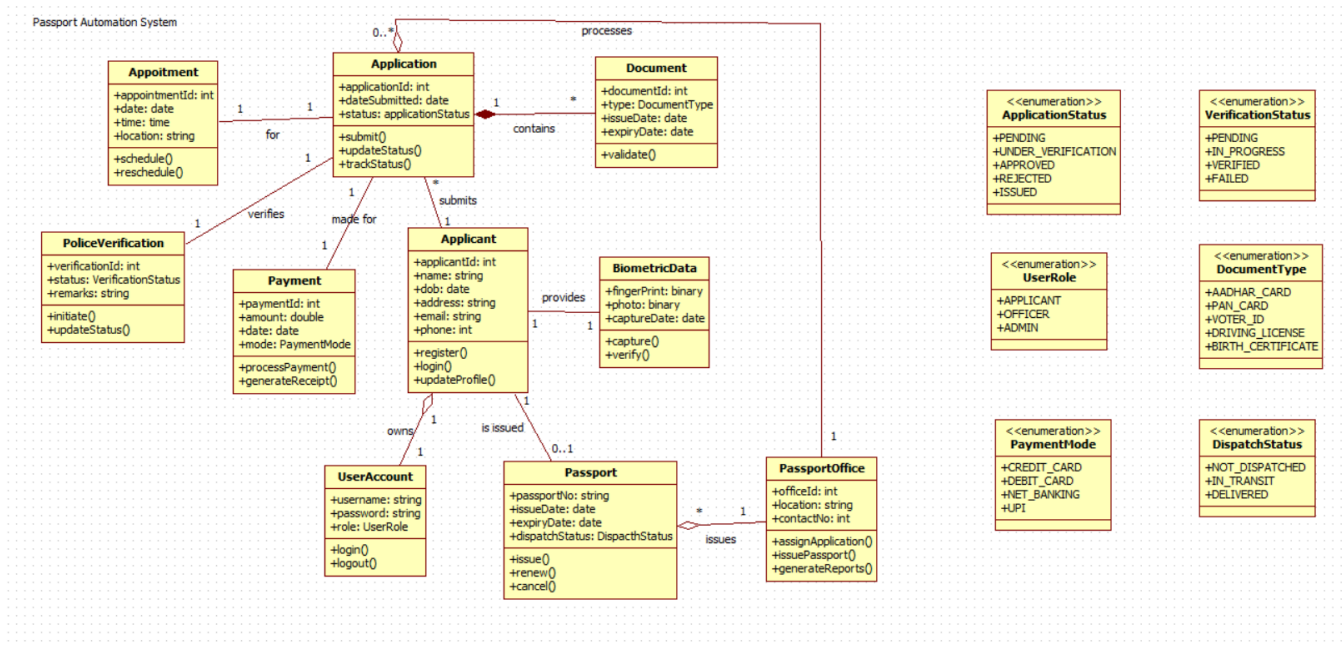
4. System Features:

- Applicant features: Apply online, upload documents, track status, schedule appointments.
- Verifier features: Check details, validate documents
- Admin features: Manage officials, applicants & generate system reports.

5. Other requirements:

- System must ensure data backup & recovery.
- Must comply with govt IT & data policies.

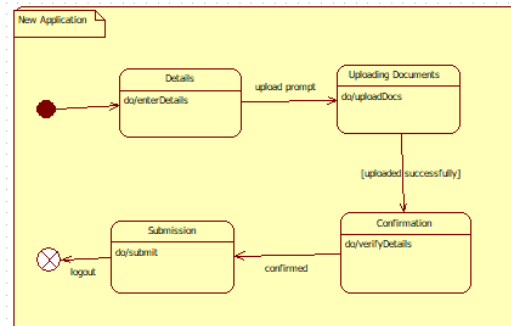
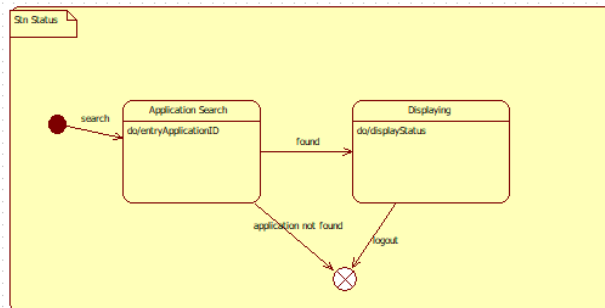
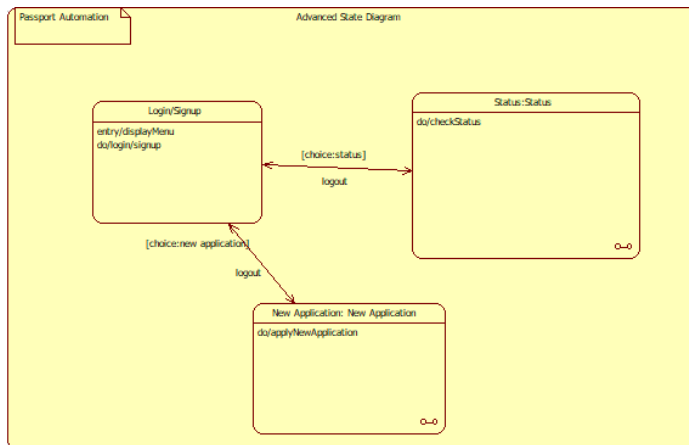
Class Diagram



Explanation:

This class diagram represents an online Passport Automation System. An Applicant (inheriting basic details from Person and Registration) fills and submits an Application, which an Administrator validates, approves, or declines. Once approved, the applicant books an Appointment for document verification. At the center, Document Verification staff verify the applicant's original documents (docType, place, etc.) during the scheduled slot. The system maintains applicant info, tracks application status, manages appointments, and logs all actions (register, update, cancel, reschedule) while ensuring only verified and approved applicants proceed to the final appointment stage.

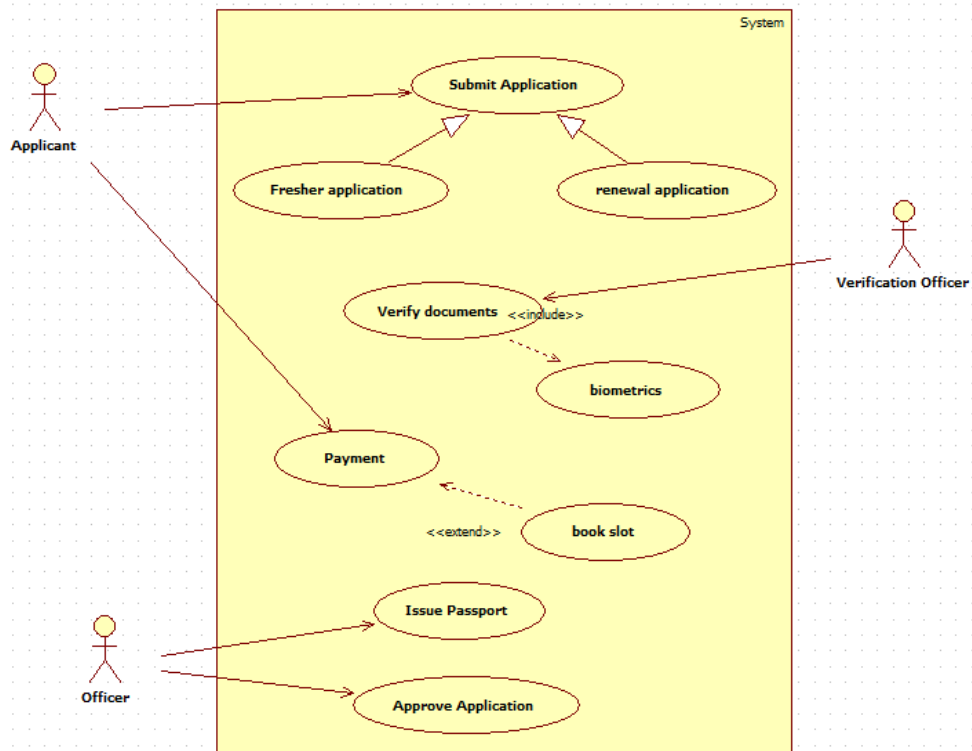
Advanced State Diagram



Advanced State:

The activity diagram presents the main menu of a Passport Automation System, where users begin by logging in or signing up. After successful authentication, they are prompted to choose between two primary options: checking the status of an existing application or starting a new passport application. The “Status” path allows users to enter their Application ID; if the record is found, the current status is displayed instantly. If not found, the process terminates gracefully with a logout. This streamlined design enables applicants to quickly track progress without re- submitting documents, enhancing user convenience and reducing support queries.

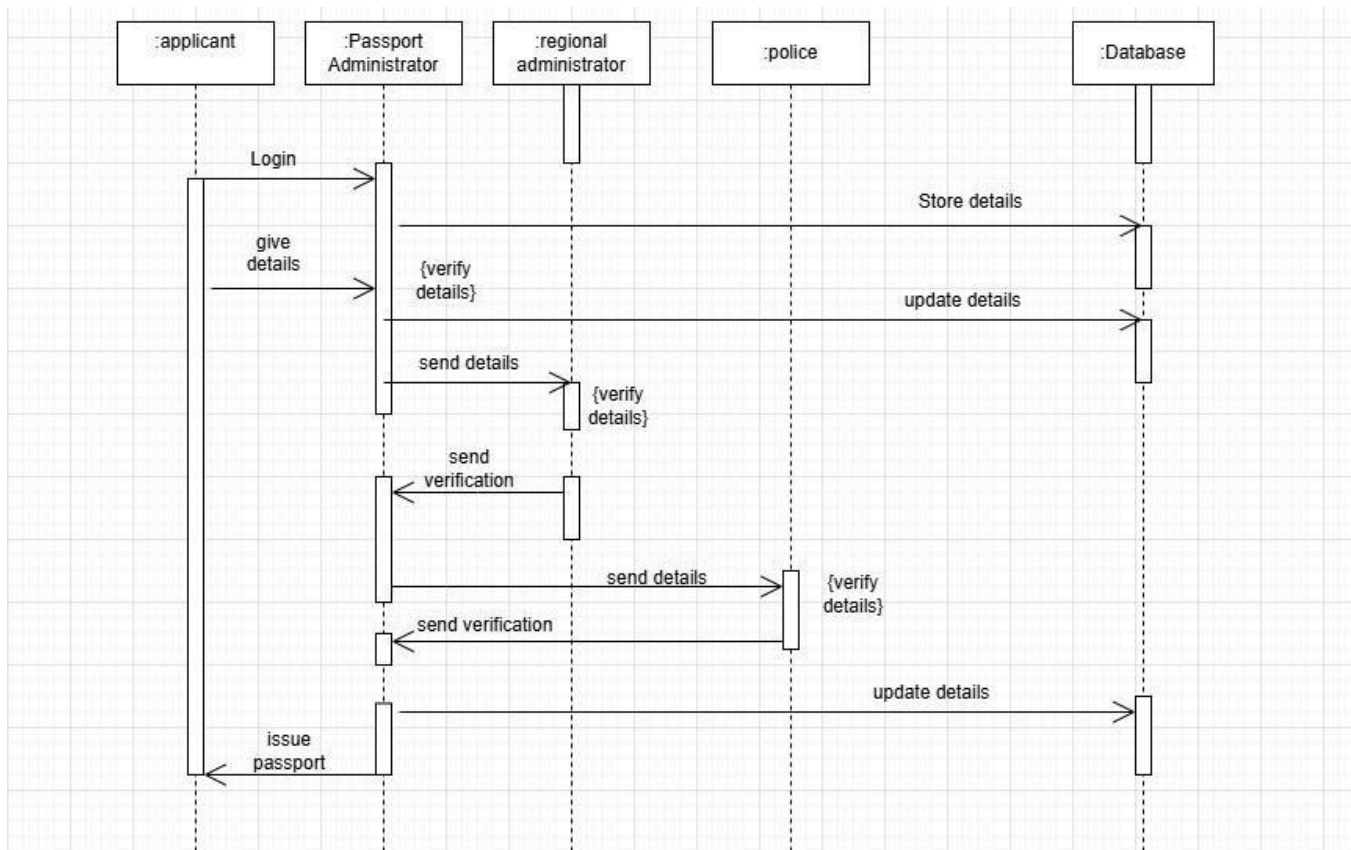
Advanced Use Case:



Advanced Use Case:

The use case diagram outlines a comprehensive Passport Automation System with four actors. The Applicant registers on the portal, logs in, fills the application form, pays fees via an external Payment Gateway, tracks status, and schedules an appointment. The "Schedule Appointment" use case includes receiving notifications and mandatory document verification. The Passport Officer verifies documents, conducts interviews, and approves or rejects the application. The Police actor performs background verification and submits the police report. This citizen-centric flow ensures online submission, transparent tracking, and seamless integration of payment and notification services.

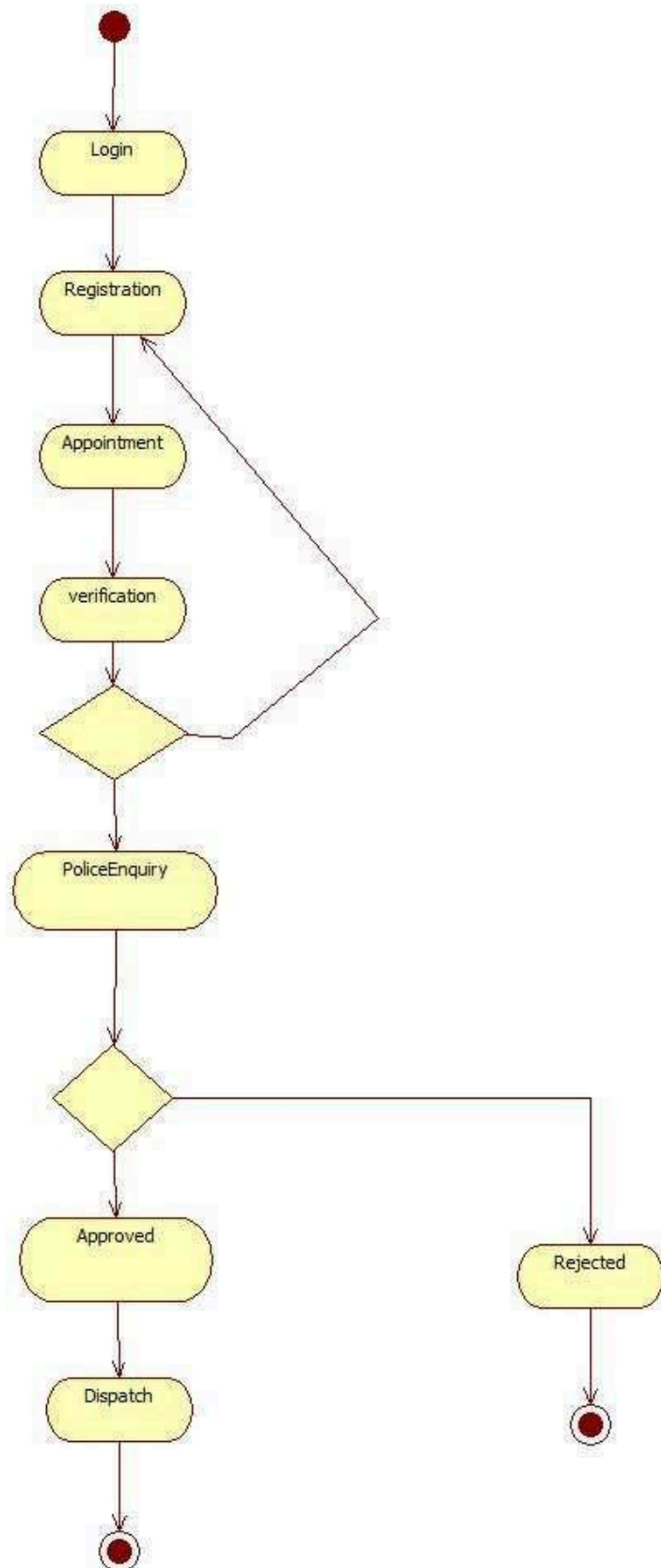
Advanced Sequence Diagram:



Advanced Sequence:

The sequence diagram depicts the complete passport application verification process. The Applicant first logs in and submits personal details to the Passport Administrator. The Administrator verifies the details, stores them in the Database, and forwards them to the Regional Administration. The Regional Administration updates the Database and sends the details to the Police for background verification. After the Police verify and return the report, the Regional Administration updates the Database again and notifies the Passport Administrator. This multi-level verification involving local, regional, and police authorities ensures thorough scrutiny of applicant information before final processing.

Advanced Activity Diagram:



Advanced Activity:

The activity diagram outlines the complete passport application lifecycle. It begins with user Login followed by Registration of personal details. The applicant then books an Appointment for document verification, after which physical Verification takes place at the passport office. A crucial decision point follows verification: if documents and identity are valid, the process continues; otherwise, it may loop back or terminate. The flow then proceeds to Police Enquiry for background and criminal record checks, ensuring eligibility. This structured sequence guarantees that only verified applicants advance to the final approval stage

